



南昌大学

课程实验报告

课程名： 计算机图形学

学 院： 数学与计算机学院 系 计算机科学与技术系

专 业： 计算机科学与技术

班 级： 计算机 201 班

学 号： 6103120034

姓 名： 龚开宸

任课教师： 徐少平

授课学期： 2022 年 ~~~~ 2023 年 上 学期

目录

实验一	安装并熟悉 OpenGL 环境.....	2
实验二	基本图形生成算法.....	7
实验三	安装并熟悉 OpenGL 环境.....	18
实验四	自由曲线绘制.....	36
实验五	OpenGL 综合实验.....	52

实验一 安装并熟悉 OpenGL 环境

一、实验目的

- 1、配置 OpenGL 环境
- 2、熟悉 OpenGL 的主要功能；
- 2、掌握 OpenGL 的绘图流程和原理；

二、实验内容

- 1、实现 OpenGL 绘图环境和 Windows 绘图环境兼容；
- 2、利用 OpenGL 绘图函数进行一些基本图形如直线、多边形的绘制；
- 3、利用 OpenGL 设置图形的颜色；
- 4、利用 OpenGL 实现键盘交互

三、实验环境

PC 微机

Windows 10 操作系统

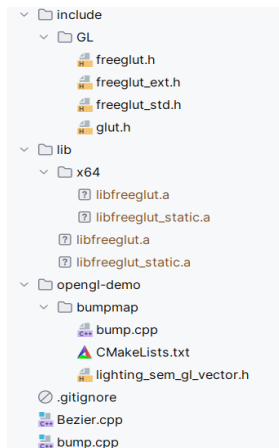
CLion 集成开发环境

四、实验步骤

- 1、采用 CLion + freeglut 搭建环境；
- 2、利用 OpenGL 绘图函数绘制三角形；
- 3、利用 OpenGL 设置图形的颜色；
- 4、利用 OpenGL 实现键盘交互

五、实验结果

1. 采用 CLion + freeglut 搭建环境。
 - (1) 从官网下载 freeglut 包
 - (2) 解压后将压缩包里的 lib 文件夹和 include 文件夹复制到自己建的工程中



(3) 修改 CMakeLists.txt 文件

```
cmake_minimum_required(VERSION 3.23)
PROJECT( tutors )

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}")

SET( PROGRAMS fog lightmaterial lightposition projection shapes texture
transformation )

SET( CXXPROGRAMS Bezier update_delta timer_func moving_car bump clone polygon_mode rotating_cube
    coordinat_sys culling depth light look_at view_port tower_rotation nuaa_cg_animation nuaa_cg_room
    lab1 lab2 lab3 lab4 lab5
    tutorial1 tutorial2 tutorial3 tutorial4 tutorial5 tutorial6)

# 1. 需要将 fog transformation.c lightposition.c transformation.c projection.c 文件
# `draw_models` 和 `screen_menu` 函数里的路径 替换为绝对路径
# 2. 需要将`glm.c`的486行的路径替换为绝对路径

include_directories(include)
link_directories(lib/x64)
```

(4) 环境搭建成功，可以开始用 OpenGL 编写程序了。

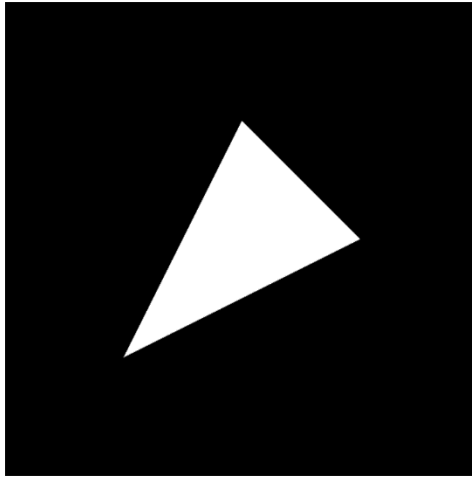
2. 利用 OpenGL 绘制简单的图形

```
void renderScene() {

    glClearColor( mask: GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);    设置显示效果

    glBegin( mode: GL_TRIANGLES);    开始绘制三角形
        glVertex3f( x: -0.5, y: -0.5, z: 0.0);
        glVertex3f( x: 0.5, y: 0.0, z: 0.0);    三角形的三个顶点
        glVertex3f( x: 0.0, y: 0.5, z: 0.0);
    glEnd();    结束绘制

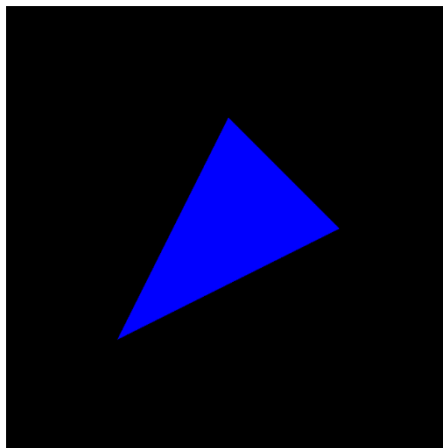
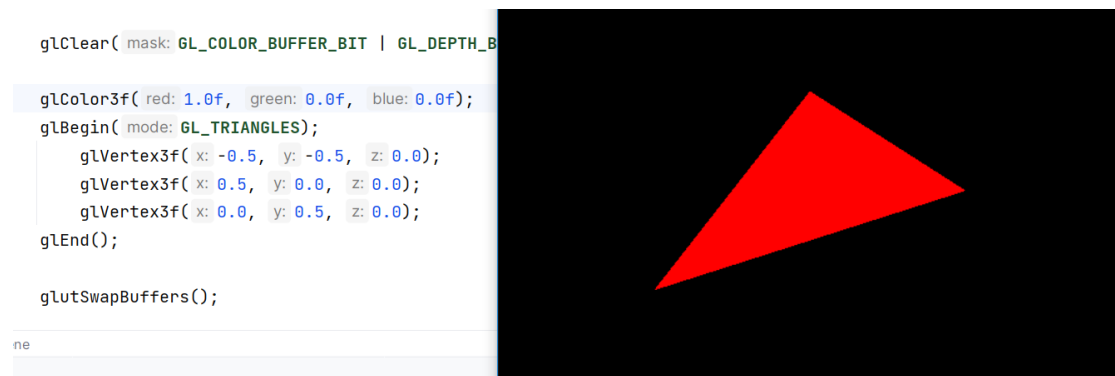
    glutSwapBuffers();    交换缓冲区
}
```



3. 利用 OpenGL 设置图形的颜色

我们可以通过 `glColor` 函数来指定图形渲染的颜色。

```
glColor3f(red, green, blue);
```



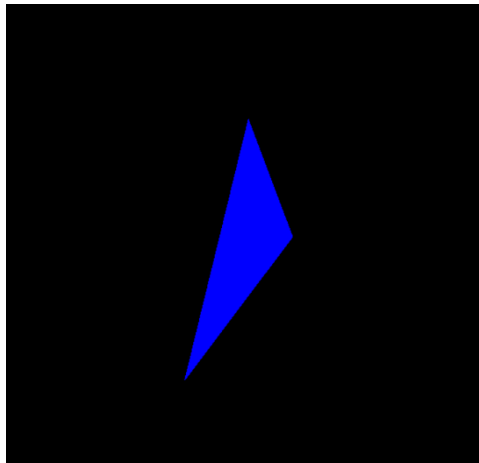
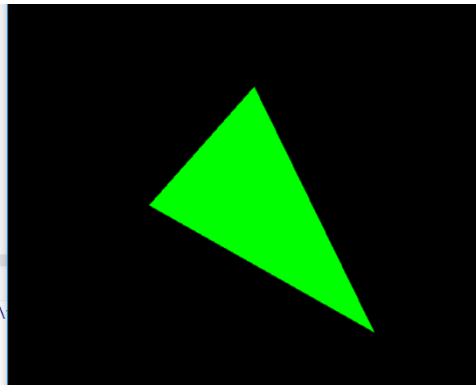
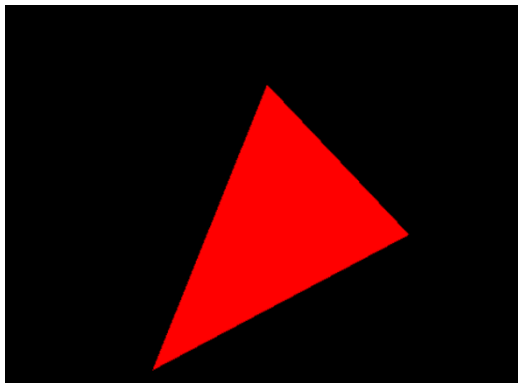
4. 利用 OpenGL 实现键盘交互

通过使用 `glutSpecialFunc` 来为键盘注册回调函数。具体实现功能为：按 F1 图形变红，按 F2，图形变绿，按住 F3 图形变蓝

```

switch (key) {
    case GLUT_KEY_F1: 红色
        red = 1.0;
        green = 0.0;
        blue = 0.0;
        break;
    case GLUT_KEY_F2: 绿色
        red = 0.0;
        green = 1.0;
        blue = 0.0;
        break;
    case GLUT_KEY_F3: 蓝色
        red = 0.0;
        green = 0.0;
        blue = 1.0;
        break;
    default:
        break;
}

```



六、实验总结

一个简单的 OpenGL 程序由四部分构成。

1. 初始化 OpenGL 和 创建窗口

```

/**
 * first step, init and create window
 */
glutInit(&argc, &argv); //init GLUT
// init window
glutInitWindowPosition(x: 250, y: 100);
glutInitWindowSize(width: 600, height: 600);
// set display mode
glutInitDisplayMode(displayMode: GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGBA);
glutCreateWindow(title: "tutors"); // create window

```

2. 显示回调函数

(1) 注册回调函数

```
glutDisplayFunc( callback: renderScene);
```

实现回调函数

```
void renderScene() {  
  
    // Clear Color and Depth Buffers  
    glClear( mask: GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
    // Reset transformations  
    glLoadIdentity();  
    // Set the camera  
    gluLookAt( eyex: 0.0f, eyey: 0.0f, eyez: 10.0f,  
               centerx: 0.0f, centery: 0.0f, centerz: 0.0f,  
               upx: 0.0f, upy: 1.0f, upz: 0.0f);  
  
    glRotatef(angle, x: 0.0f, y: 1.0f, z: 0.0f);  
  
    glColor3f(red, green, blue);  
    glBegin( mode: GL_TRIANGLES);  
        glVertex3f( x: -2.0f, y: -2.0f, z: 0.0f);  
        glVertex3f( x: 2.0f, y: 0.0f, z: 0.0f);  
        glVertex3f( x: 0.0f, y: 2.0f, z: 0.0f);  
    glEnd();  
}
```

3. 设置改变大小时的回调函数

```
void changeSize(int w, int h) {  
  
    // Prevent a divide by zero, when window is too short  
    // (you cant make a window of zero width).  
    if (h == 0)  
        h = 1;  
    float ratio = 1.0 * w / h;  
  
    // Use the Projection Matrix  
    glMatrixMode( mode: GL_PROJECTION);  
  
    // Reset Matrix  
    glLoadIdentity();  
  
    // Set the viewport to be the entire window  
    glViewport( x: 0, y: 0, width: w, height: h);  
}
```

4. 设置键盘回调函数

```
void processNormalKeys(unsigned char key, int x, int y) {  
    switch (key) {  
        case 27:  
            exit( Code: 0);  
        default:  
            break;  
    }  
}
```

实验二 基本图形生成算法

一、实验目的

- 1.掌握直线生成算法。
- 2.掌握圆和椭圆生成算法。

二、实验内容

- 1、 利用 OpenGL 绘图函数生成直线;
- 2、 利用 OpenGL 生成圆和椭圆;

三、实验环境

PC 微机

Windows 10 操作系统

CLion 集成开发环境

四、实验步骤

1. 根据 bresenham 算法, 绘制直线;
2. 根据中点画线算法, 绘制直线;
3. 根据 bresenham 画圆算法, 绘制圆;
4. 根据中点画圆算法, 绘制圆

五、实验结果

首先, 我自定义了 draw_pixel 函数

```
void draw_pixel(int ix, int iy, int value) {  
    glBegin( mode: GL_POINTS);  
    glVertex2i( x: ix, y: iy);  
    glEnd();  
}
```

1. 中点画线法

预处理步骤

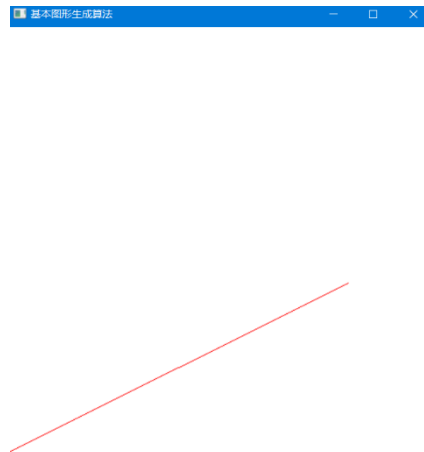
$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ len &= (\max(\Delta x, \Delta y))\end{aligned}$$

算法

```
while not in end point:
    if  $D_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + \Delta D$ 
```

```
void mid_point_draw_line(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;          delta x, delta y
    int dx = x2 - x1;
    int D = 2 * dy - dx; D_initial
    int delta_d = 2 * dy - 2 * dx; delta D
    int xk = x1, yk = y1;
    while (xk != x2 && yk != y2) { 迭代循环
        xk++;
        if (D < 0) {
            D += 2 * dy;
        } else {
            yk++;
            D += delta_d;
        }
        draw_pixel( ix: xk, iy: yk, value: BLACK);
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```

结果：



2. Bresenham 算法

前提条件

- 直线斜率 m 满足 $0 < m < 1$
- 在其他区域需要额外处理

输入

$(x_1, y_1), (x_n, y_n)$

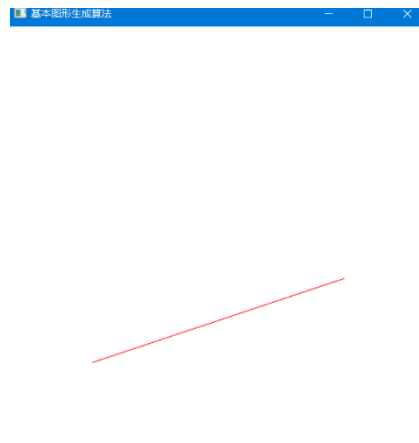
预处理步骤

$$\begin{aligned}\Delta x &= x_n - x_1 \\ \Delta y &= y_n - y_1 \\ P_{initial} &= 2\Delta y - \Delta x\end{aligned}$$

算法

```
while not in end point:
    if  $P_{initial} < 0$ 
         $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
         $D := D + 2 * \Delta y$ 
    else
         $(x_k, y_k) \rightarrow (x_k + 1, y_k + 1)$ 
         $D := D + 2 * (\Delta y - \Delta x)$ 
```

```
void bres(int x1, int y1, int x2, int y2) {
    int dy = y2 - y1;
    int dx = x2 - x1;
    int p = 2 * dy - dx;    p_initial
    int d1 = 2 * dy, d2 = 2 * (dy - dx);
    int xk = x1, yk = y1;
    int times = 0;
    while ((xk != x2 && yk != y2) || (times < dx)) {
        times++;
        xk++;
        if (p < 0) {
            p += d1;
        } else {
            p += d2;
            yk++;
        }
        draw_pixel(xk, yk, value: BLACK); 自定义绘图函数
        std::cout << "(" << xk << ", " << yk << ")" << std::endl;
    }
}
```



3. 中点画圆算法:

输入

$(x_0, y_0) R$

预处理步骤

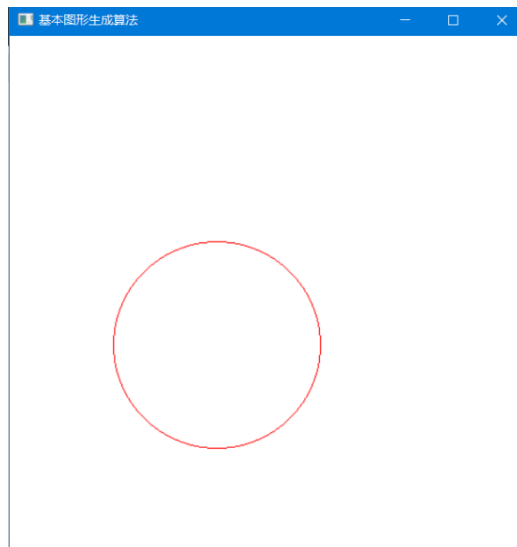
$$\begin{aligned}x_k &:= 0 \quad y_k = R \\ P_{\text{initial}} &= 1 - R\end{aligned}$$

算法

```
while  $x_c < y_c$ 
  if  $P_{\text{initial}} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 2 * x_k + 1$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 2 * y_k + 2 * x_k + 1$ 
```

```
void mid_point_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;          初始坐标
    int p = 1 - r;
    draw_whole_circle(x, y, xc, yc); drawCircle的作用是绘制 给定一点 (x,y),
    while (xc < yc) {             绘制该点在八个卦限的对应点
        xc++;
        if (p < 0) {
            p = p + 2 * xc + 1;
        } else {
            yc--;
            p = p - 2 * yc + 2 * xc + 1;
        }
        draw_whole_circle(x, y, xc, yc);
    }
}
```

```
void draw_whole_circle(int x, int y, int xc, int yc) {
    draw_pixel( ix: xc + x, iy: yc + y, value: 0);
    draw_pixel( ix: yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: -yc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: -xc + y, value: 0);
    draw_pixel( ix: -yc + x, iy: xc + y, value: 0);
    draw_pixel( ix: -xc + x, iy: yc + y, value: 0);
    std::cout << "(" << x << ", " << y << ")" << std::endl;
}
```



4. bresenham 画圆 算法

输入

(x_0, y_0) R

预处理步骤

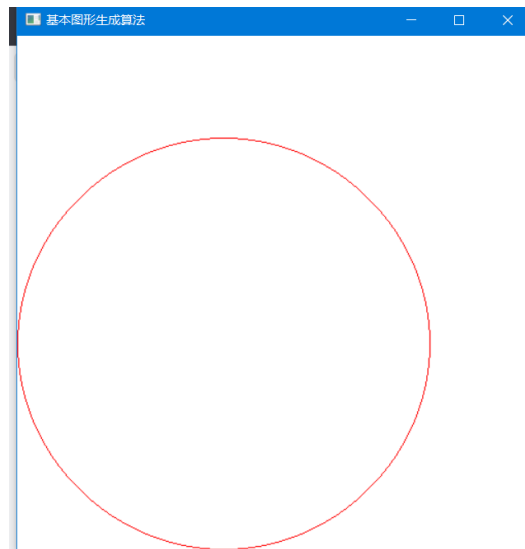
$$x_k := 0 \quad y_k = R$$

$$P_{initial} = 3 - 2R$$

算法

```
while  $x_c < y_c$ 
  if  $P_{initial} < 0$ 
     $(x_k, y_k) \rightarrow (x_k + 1, y_k)$ 
     $P := P + 4 * x_k + 6$ 
  else
     $(x_k, y_k) \rightarrow (x_k + 1, y_k - 1)$ 
     $P := P - 4 * y_k - 4 * x_k + 10$ 
```

```
void bresenham_circle_drawing(int x, int y, int r) {
    int xc = 0, yc = r;
    int p = 3 - 2 * r;
    draw_whole_circle(x, y, xc, yc);
    while (xc < yc) {
        xc++;
        if (p < 0) {
            p = p + 4 * xc + 6;
        } else {
            yc--;
            p = p + 4 * (xc - yc) + 10;
        }
        std::cout << p << " ";
        draw_whole_circle(x, y, xc, yc);
    }
    std::cout << "finish" << " ";
}
```



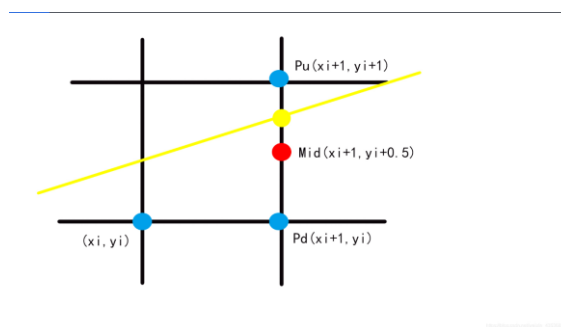
六、实验心得

1. 在本次实验中，使用中点画线法只能绘制斜率在 $0 \leq k \leq 1$ 范围内的直线，为了能够将其推广，我们先介绍中点画线算法的原理。

对于起始点 (x_0, y_0) 、结束点 (x_1, y_1) ，有 $A = y_0 - y_1$ 、 $B = x_1 - x_0$ 、 $C = (x_1 - x_0)(x_0 - y_1)$ 、 $k = (y_1 - y_0)/(x_1 - x_0)$ 。设函数 $F(x, y) = Ax + By + C$ ，则 $F(x, y)$ 将平面分为三部分：

- 1) $F(x, y) = 0$ ，点在直线上
- 2) $F(x, y) > 0$ ，点在直线的上方
- 3) $F(x, y) < 0$ ，点在直线的下方

在画线过程中，对于每个当前像素点 (x_i, y_i) ，每次在最大位移方向，即 x 方向上前进一步，在 y 方向上是否前进取决于前一格中点和直线的位置关系。



如上图所示，可判断中点 Mid 和直线的位置关系，来确定下一点的选取，令 d 为判式：

- 1) $d = F(x_i + 1, y_i + 0.5) = 0$ ， Mid 在直线上，可选取 Pu/Pd ，这里我们统一规定选取 Pd
- 2) $d = F(x_i + 1, y_i + 0.5) > 0$ ， Mid 在直线的上方，直线离 Pd 较近，选取 Pd ，则下一点 y 方向上不变
- 3) $d = F(x_i + 1, y_i + 0.5) < 0$ ， Mid 在直线的下方，直线离 Pu 较近，选取 Pu ，则下一点 y 方向上前进

利用增量思想，每下一步的 d 的判别可以基于本步 d 的正负情况，分本步 $d < 0$ 和 $d \geq 0$ 两种情况讨论。

1) $d < 0$: 本步选取 $P_u(x_i+1, y_i+1)$, 则下一步中点 $(x_i+2, y_i+1.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 1.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 1.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A + B \\ d_{new} &= d + A + B \end{aligned}$$

2) $d \geq 0$: 本步选取 $P_d(x_i+1, y_i)$, 则下一步中点 $(x_i+2, y_i+0.5)$:

$$\begin{aligned} d_{new} &= F(x_i + 2, y_i + 0.5) \\ d_{new} &= A(x_i + 2) + B(y_i + 0.5) + C \\ d_{new} &= [A(x_i + 1) + B(y_i + 0.5) + C] + A \\ d_{new} &= d + A \end{aligned}$$

在画线过程中每一点的情况都分析完了, 也易得结束画线的条件。接下来只要确定初始值就能顺利地画线了。对于起始点 (x_0, y_0) , 下一步中点为 $(x_0+1, y_0+0.5)$:

$$\begin{aligned} d_0 &= F(x_0 + 1, y_0 + 0.5) \\ d_0 &= A(x_0 + 1) + B(y_0 + 0.5) + C \\ d_0 &= [Ax_0 + By_0 + C] + A + 0.5B \\ d_0 &= A + 0.5B \end{aligned}$$

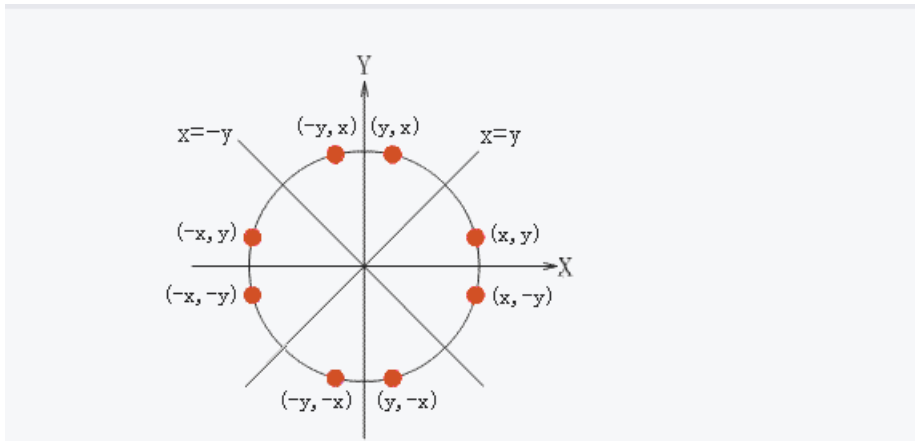
可以发现 d_0 的计算中涉及到了浮点数 0.5, 为了改进我们可以用 $2d_0$ 代替 d_0 , 因为后续计算中只考虑 d 的正负, 则扩大两倍不会对结果有所影响。此时的新 d_0 为:

$$\begin{aligned} d_0 &= 2F(x_0 + 1, y_0 + 0.5) \\ d_0 &= 2A(x_0 + 1) + 2B(y_0 + 0.5) + 2C \\ d_0 &= 2[Ax_0 + By_0 + C] + 2A + B \\ d_0 &= 2A + B \end{aligned}$$

进行推广

$k \in [0, 1]$	$k \in (1, +\infty)$	$k \in [-1, 0)$	$k \in (-\infty, -1)$
$d_0 = 2A + B$	$d_0 = A + 2B$	$d_0 = 2A - B$	$d_0 = A - 2B$
$d < 0$: $d += 2(A+B)$; $x += 1; y += 1$;	$d < 0$: $d += 2B$; $y += 1$;	$d < 0$: $d += 2A$; $x += 1$;	$d < 0$: $d += 2(A-B)$; $y -= 1; x += 1$;
$d \geq 0$: $d += 2A$; $x += 1$;	$d \geq 0$: $d += 2(A+B)$; $y += 1; x += 1$	$d \geq 0$: $d += 2(A-B)$; $x += 1; y -= 1$;	$d \geq 0$: $d -= 2B$; $y -= 1$;

2. 使用中点画圆法或者 bresenham 画圆法, 其实只能画出 $1/8$ 圆弧, 我们是利用圆的八分对称性, 画出整个圆来的。 $(x, -y)$ 、 $(-x, y)$ 、 $(-x, -y)$ 、 (y, x) 、 $(y, -x)$ 、 $(-y, x)$ 、 $(-y, -x)$



七、实验总结

这次实验主要让我们使用两种直线绘制算法（Bresenham 算法和中点画线算法）和两种圆绘制算法（Bresenham 画圆算法和中点画圆算法）绘制图形。

在实验中，我们首先学习了 Bresenham 算法和中点画线算法，然后分别使用这两种算法绘制直线。通过实验我们发现，Bresenham 算法具有更高的效率，能够更快地生成直线，而中点画线算法则更简单易懂。

接下来，我们学习了 Bresenham 画圆算法和中点画圆算法，并分别使用这两种算法绘制圆。实验结果表明，Bresenham 画圆算法和中点画圆算法都能生成圆形，但 Bresenham 画圆算法比中点画圆算法更快。

总的来说，这次实验让我更深入地了解了绘制图形的算法，并通过实际操作加深了对这些算法的理解。这对于我以后在图形学方面的学习和实践将会有很大的帮助。

八、完整代码

```
1 //
2 // Created by 1729 on 2022/11/24.
3 //
4 //A.15 Bresenham 画线算法
5
6 // for drawing a circle
7
8 #include<GL/glut.h>
9 #include <iostream>
10 #include <algorithm>
11
12 #define BLACK 1
13
14 void draw_pixel(int ix, int iy, int value) {
15     glBegin(GL_POINTS);
16     glVertex2i(ix, iy);
17     glEnd();
18 }
19
20 void draw_whole_circle(int x, int y, int xc, int yc) {
21     draw_pixel(xc + x, yc + y, 0);
```

```

22     draw_pixel(yc + x, xc + y, 0);
23     draw_pixel(yc + x, -xc + y, 0);
24     draw_pixel(xc + x, -yc + y, 0);
25     draw_pixel(-xc + x, -yc + y, 0);
26     draw_pixel(-yc + x, -xc + y, 0);
27     draw_pixel(-yc + x, xc + y, 0);
28     draw_pixel(-xc + x, yc + y, 0);
29     std::cout << "(" << x << ", " << y << ")" << std::endl;
30 }
31
32 void bresenham_circle_drawing(int x, int y, int r) {
33     int xc = 0, yc = r;
34     int p = 3 - 2 * r;
35     draw_whole_circle(x, y, xc, yc);
36     while (xc < yc) {
37         xc++;
38         if (p < 0) {
39             p = p + 4 * xc + 6;
40         } else {
41             yc--;
42             p = p + 4 * (xc - yc) + 10;
43         }
44         std::cout << p << " ";
45         draw_whole_circle(x, y, xc, yc);
46     }
47     std::cout << "finish" << " ";
48 }
49
50 void mid_point_circle_drawing(int x, int y, int r) {
51     int xc = 0, yc = r;
52     int p = 1 - r;
53     draw_whole_circle(x, y, xc, yc);
54     while (xc < yc) {
55         xc++;
56         if (p < 0) {
57             p = p + 2 * xc + 1;
58         } else {
59             yc--;
60             p = p - 2 * yc + 2 * xc + 1;
61         }
62         draw_whole_circle(x, y, xc, yc);
63     }
64 }
65

```



```

66 void mid_point_draw_line(int x1, int y1, int x2, int y2) {
67     int dy = y2 - y1;
68     int dx = x2 - x1;
69     int D = 2 * dy - dx;
70     int delta_d = 2 * dy - 2 * dx;
71     int xk = x1, yk = y1;
72     while (xk != x2 && yk != y2) {
73         xk++;
74         if (D < 0) {
75             D += 2 * dy;
76         } else {
77             yk++;
78             D += delta_d;
79         }
80         draw_pixel(xk, yk, BLACK);
81         std::cout << "(" << xk << ", " << yk << ")" << std::endl;
82     }
83 }
84
85 void bres(int x1, int y1, int x2, int y2) {
86     int dy = y2 - y1;
87     int dx = x2 - x1;
88     int p = 2 * dy - dx;
89     int d1 = 2 * dy, d2 = 2 * (dy - dx);
90     int xk = x1, yk = y1;
91     int times = 0;
92     while ((xk != x2 && yk != y2) || (times < dx)) {
93         times++;
94         xk++;
95         if (p < 0) {
96             p += d1;
97         } else {
98             p += d2;
99             yk++;
100         }
101         draw_pixel(xk, yk, BLACK);
102         std::cout << "(" << xk << ", " << yk << ")" << std::endl;
103     }
104 }
105
106 void display() {
107     glClear(GL_COLOR_BUFFER_BIT);
108     // bres(100,100,400,200);
109     // mid_point_draw_line(2, 2, 400, 200);

```

```

110 //    mid_point_circle_drawing(200, 200, 100);
111    bresnanham_circle_drawing(200, 200, 200);
112    glFlush();
113 }
114
115 void myinit() {
116     glClearColor(1.0, 1.0, 1.0, 1.0);
117     glColor3f(1.0, 0.0, 0.0);
118     glPointSize(1.0);
119     glMatrixMode(GL_PROJECTION);
120     glLoadIdentity();
121     gluOrtho2D(0.0, 499.0, 0.0, 499.0);
122 }
123
124 int main(int argc, char **argv) {
125     /* standard GLUT initialization */
126     glutInit(&argc, argv);
127     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); /*default, not
needed*/
128     glutInitWindowSize(500, 500); /* 500×500 pixel window */
129     glutInitWindowPosition(0, 0); /* place window top left on
display */
130     glutCreateWindow("基本图形生成算法"); /* window title */
131     glutDisplayFunc(display); /* display callback invoked when
window
132                                opened
133                                */
134     myinit(); /*set attributes set background */
135     glutMainLoop(); /* enter event loop */
136

```

实验三 安装并熟悉 OpenGL 环境

一、实验目的

1. 熟悉 Cohen-Sutherland 算法;
2. 熟悉中点分割算法;
3. 熟悉 Liang-Barskey 算法

二、实验内容

- 1、实现 OpenGL 绘图环境和 Windows 绘图环境兼容;
- 2、利用 OpenGL 实现 Cohen-Sutherland 算法;
- 3、利用 OpenGL 实现中点分割算法;
- 4、利用 OpenGL 实现 Liang-Barskey 算法

三、实验环境

PC 微机

Windows 10 操作系统

CLion 集成开发环境

四、实验步骤

- 1、根据 Cohen-Sutherland 算法对直线进行裁切;
- 2、根据中点分割算法对直线进行裁切;
- 3、根据 Liang-Barskey 算法对直线进行裁切

五、实验结果

0、本实验采用交互式设计，用户可以使用鼠标绘制直线，或者使用右键取消绘制。绘制完直线后，按 c 键进行直线裁切。

- 1、根据 Cohen-Sutherland 算法对直线进行裁切;

使用 Cohen-Sutherland 的步骤

将矩形的边划分为四个部分：左边、右边、上边、下边。

定义一个四位二进制数，其中每一位分别表示该点是否在矩形的左边、右边、上边、下边的一侧。

遍历线段的两个端点，对每一个端点进行如下操作：

- a. 如果该端点在矩形内，该端点的二进制值为 0000。
- b. 如果该端点在矩形的左边上，该端点的二进制值为 0001。
- c. 如果该端点在矩形的右边上，该端点的二进制值为 0010。

d. 如果该端点在矩形的上边上, 该端点的二进制值为 0100。

e. 如果该端点在矩形的下边上, 该端点的二进制值为 1000。

根据线段的两个端点的二进制值, 判断它们是否相交:

a. 如果两个端点的二进制值进行 OR 操作后, 值为 0, 那么直线必然在矩形内部

b. 如果两个端点的二进制值进行 XOR 操作后, 值不为 0, 那么直线必然在矩形外部

c. 如果不符合以上两者, 那么直线就可能和矩形相交。我们对矩形的左、右、下、上分别讨论

(1). 将端点的二进制与 0x01 (左侧) 按位与, 若结果大于 0, 则相交。我们用斜率计算公式计算新的端点

(2). 将端点的二进制与 0x02 (右侧) 按位与, 若结果大于 0, 则相交。我们用斜率计算公式计算新的端点

(3). 将端点的二进制与 0x04 (上侧) 按位与, 若结果大于 0, 则相交。我们用斜率计算公式计算新的端点

(4). 将端点的二进制与 0x08 (下侧) 按位与, 若结果大于 0, 则相交。我们用斜率计算公式计算新的端点

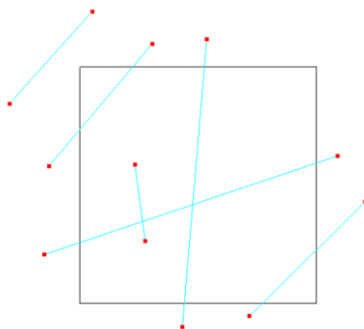
$$y = y_0 + m(x - x_0) \quad (14)$$

where the x value is set to either $xw_{\min}$ or $xw_{\max}$, and the slope of the line is calculated as $m = (y_{\text{end}} - y_0)/(x_{\text{end}} - x_0)$. Similarly, if we are looking for the intersection with a horizontal border, the x coordinate can be calculated as

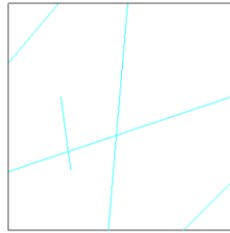
$$x = x_0 + \frac{y - y_0}{m} \quad (15)$$

结果示意图:

截取前:



截取后:



代码实现

```
while (!done) { // 状态机
    code1 = encode( pt: p1, winMin, winMax);
    code2 = encode( pt: p2, winMin, winMax);
    if (accept(code1, code2)) {
        done = true;
        plotLine = true;
    } else if (reject(code1, code2))
        done = true;
    else {
        if (inside( code: code1)) {
            swapPts( p1: &p1, p2: &p2);
            swapCodes( c1: &code1, c2: &code2);
        }
        /* Use slope m to find line-clipEdge intersection. */
        if (p2.x != p1.x)
            m = (p2.y - p1.y) / (p2.x - p1.x);
        if (code1 & winLeftBitCode) { // 裁切掉 区域左边界 以左 的线段
            p1.y += (winMin.x - p1.x) * m;
            p1.x = winMin.x;
        } else if (code1 & winRightBitCode) { // 裁切掉 区域左边界 以右 的线段
            p1.y += (winMax.x - p1.x) * m;
            p1.x = winMax.x;
        } else if (code1 & winBottomBitCode) {
            if (p2.x != p1.x)
                p1.x += (winMin.y - p1.y) / m;
```

2、 根据中点分割算法对直线进行裁切；

中点分割算法与 Cohen-Sutherland 裁切算法非常相似，仅在处理 c 类情况时有所区别

c. 如果不符合以上两者，那么直线就可能和矩形相交。我们取直线端的中点 mid，如果中点在矩形内部，我们令 $p1 = mid$ ，否则， $p2 = mid$ 。更新后，递归判断 $p1p2$ 与矩形的位置关系。

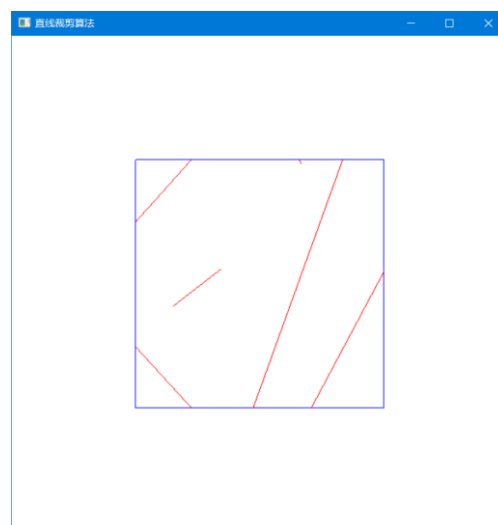
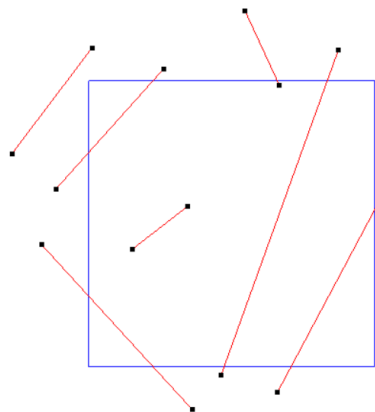
代码实现：

```

void clipLine(GLint winMin, GLint winMax, GLint p1, GLint p2) {
    GLubyte code1, code2;
    GLint done = false, plotLine = false;
    GLfloat m;
    while (!done) { // 状态机
        code1 = encode(pt: p1, winMin, winMax);
        code2 = encode(pt: p2, winMin, winMax);
        if (accept(code1, code2)) {
            done = true;
            plotLine = true;
        } else if (reject(code1, code2)) {
            done = true;
        } else {
            /* Label the endpoint outside the display window as p1. */
            if (inside(code: code1)) {
                swapPts(p1: &p1, p2: &p2);
                swapCodes(c1: &code1, c2: &code2);
            } else {
                wPt2D mid = { .x: (p1.x + p2.x) / 2, .y: (p1.y + p2.y) / 2 };
                auto code3: GLubyte = encode(pt: mid, winMin, winMax);
                if (inside(code: code3)) {
                    p1 = mid;
                } else {
                    if ((code3 ^ code2) == 0x11) {
                        p2 = mid;
                    }
                }
            }
        }
    }
}

```

运行结果



3、根据 Liang-Barskey 算法对直线进行裁切

Liang-Barskey 针对 Cohen-Sutherland 进行了较大改进。

首先它提出我们可以用参数方程来表达直线

$$\begin{aligned} x &= x_0 + u\Delta x \\ y &= y_0 + u\Delta y \quad 0 \leq u \leq 1 \end{aligned} \quad (16)$$

那么我们要求的问题就转化为使得直线落在矩形范围内的 u 的取值范围。

$$\begin{aligned} xw_{\min} &\leq x_0 + u\Delta x \leq xw_{\max} \\ yw_{\min} &\leq y_0 + u\Delta y \leq yw_{\max} \end{aligned} \quad (17)$$

我们可以将这两个不等式一般化

which can be expressed as

$$u p_k \leq q_k, \quad k = 1, 2, 3, 4 \quad (18)$$

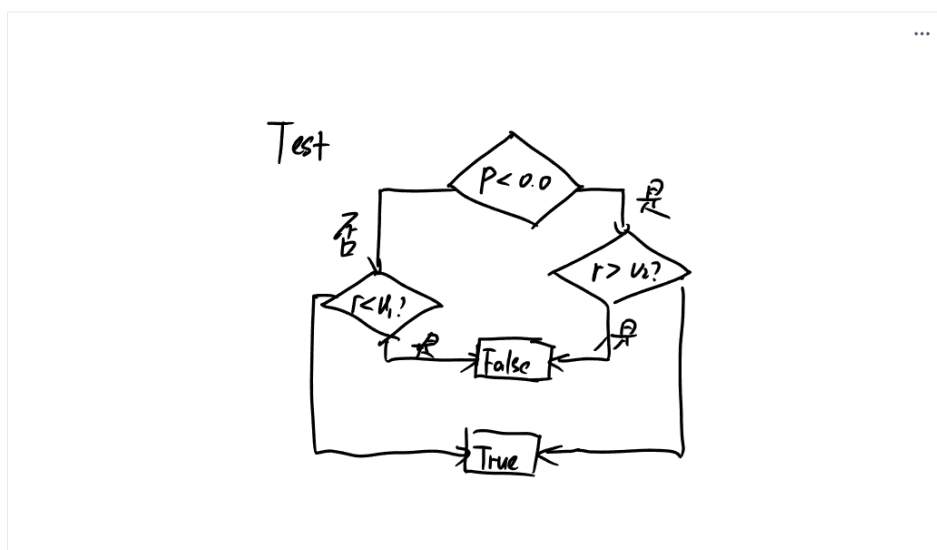
where parameters p and q are defined as

$$\begin{aligned} p_1 &= -\Delta x, & q_1 &= x_0 - xw_{\min} \\ p_2 &= \Delta x, & q_2 &= xw_{\max} - x_0 \\ p_3 &= -\Delta y, & q_3 &= y_0 - yw_{\min} \\ p_4 &= \Delta y, & q_4 &= yw_{\max} - y_0 \end{aligned} \quad (19)$$

这样，根据四个不等式，我们就可以求解出 u 的范围。

$$r_i = \frac{q_i}{p_i}, u_1 = \max\{0, r_i\}, u_2 = \min\{1, r_i\}$$

更新 u_1, u_2 的算法如下



```

GLint clipTest(GLfloat p, GLfloat q, GLfloat *u1, GLfloat *u2) {
    GLfloat r;
    GLint returnValue = true;
    if (p < 0.0) {
        r = q / p;
        if (r > *u2)
            returnValue = false;
        else if (r > *u1)
            *u1 = r;
    } else if (p > 0.0) {
        r = q / p;
        if (r < *u1)
            returnValue = false;
        else if (r < *u2)
            *u2 = r;
    } else
        /* Thus p = 0 and line is parallel to clipping boundary. */
        if (q < 0.0)
            /* Line is outside clipping boundary. */
            returnValue = false;
    return returnValue;
}

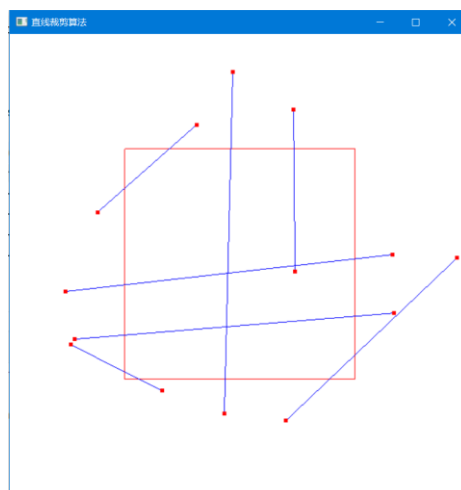
```

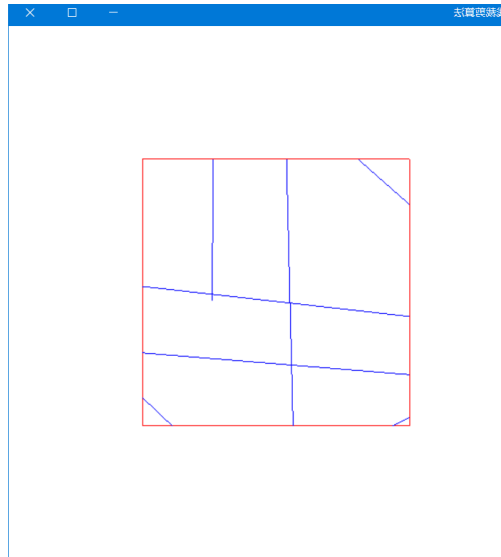
更新完 u1, u2 的值后，就可以开始绘制直线了

```

void lineClipLiangBarsk(wcPt2D winMin, wcPt2D winMax, wcPt2D p1, wcPt2D p2) {
    GLfloat u1 = 0.0, u2 = 1.0, dx = p2.getx() - p1.getx(), dy;
    if (clipTest(p1.getx() - winMin.getx(), u1: &u1, u2: &u2))
        if (clipTest(p2.getx() - winMax.getx() - p1.getx(), u1: &u1, u2: &u2)) {
            dy = p2.gety() - p1.gety();
            if (clipTest(p1.gety() - winMin.gety(), u1: &u1, u2: &u2))
                if (clipTest(p2.gety() - winMax.gety() - p1.gety(), u1: &u1, u2: &u2)) {
                    if (u2 < 1.0) {
                        p2.setCoords(p1.getx() + u2 * dx, p1.gety() + u2 * dy);
                    }
                    if (u1 > 0.0) {
                        p1.setCoords(p1.getx() + u1 * dx, p1.gety() + u1 * dy);
                    }
                    setLine(p1.getx(), p1.gety(), p2.getx(), p2.gety());
                }
        }
}

```





六、实验心得

1. 一开始向 openGL 的窗口画点的时候，看不到任何效果。调试了许久才发现是因为一个点太小了，在屏幕上看不清楚，我们可以通过 `glPointSize` 来调整大小。
2. 我们可以使用 `glut_down` 来检测鼠标是否按下，`glut_left_button` or `glut_right_button` 来判断是鼠标左键还是右键。
3. 在用户输入导致窗口状态变更后，我们可以直接调用 `glutPostRedisplay` 来重新绘制窗口
4. 这次实验主要让我们使用三种直线裁切算法（Cohen-Sutherland 算法、中点裁切算法和 Liang-Barskey 算法）对直线进行裁切。

在实验中，我们首先学习了 Cohen-Sutherland 算法、中点裁切算法和 Liang-Barskey 算法，然后分别使用这三种算法对直线进行裁切。通过实验我们发现，Cohen-Sutherland 算法具有高效简单的特点，能够快速准确地对直线进行裁切；中点裁切算法的操作简单，但效率不如 Cohen-Sutherland 算法；Liang-Barskey 算法效率较高，但实现起来相对比较困难。

总的来说，这次实验让我更深入地了解了直线裁切的算法，并通过实际操作加深了对这些算法的理解。这对于我以后在图形学方面的学习和实践将会有很大的帮助。

七、完整代码

```
1 #include <GL/glut.h>
2 #include <vector>
3 #include <iostream>
4 #include <set>
5 #include <cmath>
6
7 using std::vector;
8 using std::set;
9 using std::cout;
```

```

10 using std::endl;
11 const double eps = 1e-6;
12 #define BLACK 0.0, 0.0, 0.0
13 #define WHITE 1.0, 1.0, 1.0
14 #define RED 1.0, 0.0, 0.0
15 #define GREEN 0.0, 1.0, 0.0
16 #define BLUE 0.0, 0.0, 1.0
17 #define YELLOW 1.0, 1.0, 0.0
18 #define CYAN 0.0, 1.0, 1.0
19 #define MAGENTA 1.0, 0.0, 1.0
20 #define ORANGE 1.0, 0.5, 0.0
21
22 static bool equalForDouble(double f1, double f2) {
23     return fabs(f1 - f2) <= eps;
24 };
25
26 class wcPt2D {
27 public:
28     GLfloat x, y;
29
30     void setCoords(GLfloat xCoord, GLfloat yCoord) {
31         x = xCoord;
32         y = yCoord;
33     }
34
35     GLfloat getX() const {
36         return x;
37     }
38
39     GLfloat getY() const {
40         return y;
41     }
42
43     friend std::ostream &operator<<(std::ostream &out, const
wcPt2D &p) {
44         out << "(" << p.x << ", " << p.y << ")";
45     }
46
47     bool operator<(const wcPt2D &other) const {
48         return x < other.x && y < other.y;
49     }
50
51     bool operator==(const wcPt2D &other) const {

```

```

52         return equalForDouble(x, other.x) && equalForDouble(y,
other.y);
53     }
54 };
55
56 class wcLine2D {
57 public:
58     wcPt2D p1{}, p2{};
59     int cnt;
60
61     wcLine2D() {
62         cnt = 0;
63     }
64
65     bool drawingPrepared() {
66         return cnt == 2;
67     }
68
69     void addPoint(float x, float y) {
70         if (cnt == 0) {
71             p1 = {x, y};
72         } else if (cnt == 1) {
73             p2 = {x, y};
74         }
75         cnt++;
76     }
77
78     wcPt2D getMidPoint () {
79         wcPt2D p = {(p1.x + p2.x) / 2, (p1.y + p2.y) / 2};
80         return p;
81     }
82
83     friend std::ostream &operator<<(std::ostream &out, const
wcLine2D &l) {
84         out << "[" << l.p1 << " " << l.p2 << "]";
85     }
86 };
87
88 const unsigned WIDTH = 600;
89 const unsigned HEIGHT = 600;
90 const unsigned int MAX_POINT = 2;
91 const unsigned int MAX_LINE = 10;
92 vector<wcLine2D> lines = vector<wcLine2D>();
93 //set<wcPt2D> points = set<wcPt2D>();

```

```

94 vector<wcPt2D> points = vector<wcPt2D>();
95 wcLine2D curLine = wcLine2D();
96 wcPt2D WINMIN = {-0.5, -0.5};
97 wcPt2D WINMAX = {0.5, 0.5};
98 bool clip = false;
99
100 void renderScene();
101
102 void setLine(wcPt2D p1, wcPt2D p2);
103
104 void setLine(float x1, float y1, float x2, float y2);
105
106 void setPoint(wcPt2D p) {
107     glColor3f(RED);
108     glPointSize(5);
109     glBegin(GL_POINTS);
110     glVertex2f(p.x, p.y);
111     glEnd();
112     glFlush();
113 }
114
115 void setLine(wcPt2D p1, wcPt2D p2) {
116     glColor3f(BLUE);
117     glBegin(GL_LINES);
118     glVertex2f(p1.x, p1.y);
119     glVertex2f(p2.x, p2.y);
120     glEnd();
121     glFlush();
122 }
123
124 void setLine(float x1, float y1, float x2, float y2) {
125     wcPt2D p1 = {x1, y1};
126     wcPt2D p2 = {x2, y2};
127     setLine(p1, p2);
128 }
129
130 inline GLint round(const GLfloat a) { return GLint(a + 0.5); }
131
132 /* Define a four-bit code for each of the outside regions of a
133 * rectangular clipping window.
134 */
135 const GLint winLeftBitCode = 0x1;
136 const GLint winRightBitCode = 0x2;
137 const GLint winBottomBitCode = 0x4;

```

```

138 const GLint winTopBitCode = 0x8;
139
140 /* A bit-mask region code is also assigned to each endpoint of an
input
141 * line segment, according to its position relative to the four
edges of
142 * an input rectangular clip window.
143 *
144 * An endpoint with a region-code value of 0000 is inside the
clipping
145 * window, otherwise it is outside at least one clipping boundary.
If
146 * the 'or' operation for the two endpoint codes produces a value
of
147 * false, the entire line defined by these two endpoints is saved
148 * (accepted). If the 'and' operation between two endpoint codes
is
149 * true, the line is completely outside the clipping window, and
it is
150 * eliminated (rejected) from further processing.
151 */
152 inline GLint inside(GLint code) { return GLint(!code); }
153
154 inline GLint reject(GLint code1, GLint code2) { return
GLint(code1 & code2); }
155
156 inline GLint accept(GLint code1, GLint code2) { return
GLint(!(code1 | code2)); }
157
158 GLubyte encode(wcPt2D pt, wcPt2D winMin, wcPt2D winMax) {
159     GLubyte code = 0x00;
160     if (pt.x < winMin.x)
161         code = code | winLeftBitCode;
162     if (pt.x > winMax.x)
163         code = code | winRightBitCode;
164     if (pt.y < winMin.y)
165         code = code | winBottomBitCode;
166     if (pt.y > winMax.y)
167         code = code | winTopBitCode;
168     return (code);
169 }
170
171 void swapPts(wcPt2D *p1, wcPt2D *p2) {
172     wcPt2D tmp;

```

```

173     tmp = *p1;
174     *p1 = *p2;
175     *p2 = tmp;
176 }
177
178 void swapCodes(GLubyte *c1, GLubyte *c2) {
179     GLubyte tmp;
180     tmp = *c1;
181     *c1 = *c2;
182     *c2 = tmp;
183 }
184
185
186 void processNormalKeys(unsigned char key, int x, int y) {
187     switch (key) {
188         //Q 键退出程序
189         case 'q':
190         case 'Q':
191             exit(0);
192             break;
193         //C 键重绘
194         case 'c':
195         case 'C':
196             clip = true;
197             glutPostRedisplay();
198             break;
199         //R 键撤销一个点
200         case 'r':
201         case 'R':
202             // numberOfControlPoints--;
203             glutPostRedisplay();
204             break;
205     }
206 }
207
208 void mouse(int button, int state, int x, int y) {
209     // 点击鼠标一次，但是却打印了两次
210     float wx, wy;
211
212     if (state == GLUT_DOWN) {
213         //鼠标左键为绘制
214         if (button == GLUT_LEFT_BUTTON) {
215             //判断控制点数目是否超过最大值
216             wx = (2.0 * x) / (float) (WIDTH - 1) - 1.0;

```

```

217         wy = (2.0 * (HEIGHT - 1 - y)) / (float) (HEIGHT - 1) -
1.0;
218         curLine.addPoint(wx, wy);
219         points.push_back({wx, wy});
220         if (curLine.drawingPrepared()) {
221             cout << "adding Line " << curLine << endl;
222             lines.push_back(curLine);
223             curLine = wcLine2D();
224         }
225         renderScene();
226     }
227     //非编辑状态下, 右键点选后, 将待修改的点进行标记
228     if (button == GLUT_RIGHT_BUTTON) {
229         lines.pop_back();
230         points.pop_back();
231         points.pop_back();
232     }
233 }
234 }
235
236 GLint clipTest(GLfloat p, GLfloat q, GLfloat *u1, GLfloat *u2) {
237     GLfloat r;
238     GLint returnValue = true;
239     if (p < 0.0) {
240         r = q / p;
241         if (r > *u2)
242             returnValue = false;
243         else if (r > *u1)
244             *u1 = r;
245     } else if (p > 0.0) {
246         r = q / p;
247         if (r < *u1)
248             returnValue = false;
249         else if (r < *u2)
250             *u2 = r;
251     } else
252     /* Thus p = 0 and line is parallel to clipping boundary. */
253     if (q < 0.0)
254     /* Line is outside clipping boundary. */
255         returnValue = false;
256     return returnValue;
257 }
258

```

```

259 void lineClipLiangBarsk(wcPt2D winMin, wcPt2D winMax, wcPt2D p1,
wcPt2D p2) {
260     GLfloat u1 = 0.0, u2 = 1.0, dx = p2.getx() - p1.getx(), dy;
261     if (clipTest(-dx, p1.getx() - winMin.getx(), &u1, &u2))
262         if (clipTest(dx, winMax.getx() - p1.getx(), &u1, &u2)) {
263             dy = p2.gety() - p1.gety();
264             if (clipTest(-dy, p1.gety() - winMin.gety(), &u1, &u2))
265                 if (clipTest(dy, winMax.gety() - p1.gety(), &u1,
&u2)) {
266                     if (u2 < 1.0) {
267                         p2.setCoords(p1.getx() + u2 * dx, p1.gety() +
u2 * dy);
268                     }
269                     if (u1 > 0.0) {
270                         p1.setCoords(p1.getx() + u1 * dx, p1.gety() +
u1 * dy);
271                     }
272                     setLine(p1.getx(), p1.gety(),
p2.getx(), p2.gety());
273                 }
274             }
275     }
276 }
277
278 void lineClipCohSuth(wcPt2D winMin, wcPt2D winMax, wcPt2D p1,
wcPt2D p2) {
279     GLubyte code1, code2;
280     GLint done = false, plotLine = false;
281     GLfloat m;
282     while (!done) { // 状态机
283         code1 = encode(p1, winMin, winMax);
284         code2 = encode(p2, winMin, winMax);
285         if (accept(code1, code2)) {
286             done = true;
287             plotLine = true;
288         } else if (reject(code1, code2))
289             done = true;
290         else {
291             if (inside(code1)) {
292                 swapPts(&p1, &p2);
293                 swapCodes(&code1, &code2);
294             }
295             /* Use slope m to find line-clipEdge intersection. */
296             if (p2.x != p1.x)
297                 m = (p2.y - p1.y) / (p2.x - p1.x);

```



```

298         if (code1 & winLeftBitCode) { // 裁切掉 区域左边界 以左 的线
段
299             p1.y += (winMin.x - p1.x) * m;
300             p1.x = winMin.x;
301         } else if (code1 & winRightBitCode) { // 裁切掉 区域右边界
以右 的线段
302             p1.y += (winMax.x - p1.x) * m;
303             p1.x = winMax.x;
304         } else if (code1 & winBottomBitCode) {
305             if (p2.x != p1.x)
306                 p1.x += (winMin.y - p1.y) / m;
307             p1.y = winMin.y;
308         } else if (code1 & winTopBitCode) {
309             if (p2.x != p1.x)
310                 p1.x += (winMax.y - p1.y) / m;
311             p1.y = winMax.y;
312         }
313     }
314 }
315 if (plotLine) {
316     setLine((p1.x), (p1.y), (p2.x), (p2.y));
317 }
318 }
319
320
321 void lineClipMidPoint(wcPt2D winMin, wcPt2D winMax, wcPt2D p1,
wcPt2D p2) {
322     GLubyte code1, code2;
323     GLint done = false, plotLine = false;
324     GLfloat m;
325     while (!done) { // 状态机
326         code1 = encode(p1, winMin, winMax);
327         code2 = encode(p2, winMin, winMax);
328         if (accept(code1, code2)) {
329             done = true;
330             plotLine = true;
331         } else if (reject(code1, code2))
332             done = true;
333         else {
334             /* Label the endpoint outside the display window as p1. */
335             if (inside(code1)) {
336                 swapPts(&p1, &p2);
337                 swapCodes(&code1, &code2);
338             } else {

```

```

339         wcPt2D mid = {(p1.x + p2.x) / 2 , (p1.y + p2.y) / 2};
340         auto code3 = encode(mid, winMin, winMax);
341         if (inside(code3)) {
342             p1 = mid;
343         } else {
344             if ((code3 ^ code2) == 0x11) {
345                 p2 = mid;
346             }
347         }
348     }
349 }
350
351 }
352 if (plotLine) {
353     setLine((p1.x), (p1.y), (p2.x), (p2.y));
354 }
355 }
356
357 void drawLines() {
358     if (lines.empty()) {
359         return;
360     }
361     for (auto line: lines) {
362         setLine(line.p1, line.p2);
363     }
364 }
365
366 void drawPoints() {
367     if (points.empty()) {
368         return;
369     }
370
371     for (auto p: points) {
372         setPoint(p);
373     }
374 }
375
376 void drawQuads() {
377
378     glColor3f(RED);
379     glBegin(GL_LINE_LOOP);
380     glVertex2f(-0.5, -0.5);
381     glVertex2f(-0.5, 0.5);
382     glVertex2f(0.5, 0.5);

```

```

383     glVertex2f(0.5, -0.5);
384     glEnd();
385 }
386
387 void renderScene() {
388     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
389     // set the viewport
390     glViewport(0, 0, 600, 600);
391
392     // set the projection matrix
393     glMatrixMode(GL_PROJECTION);
394     glLoadIdentity();
395     glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
396
397     // set the modelview matrix
398     glMatrixMode(GL_MODELVIEW);
399     glLoadIdentity();
400
401     // ++++++
402     drawQuads();
403     if (!clip) {
404         drawLines();
405         drawPoints();
406     } else {
407         for (auto line: lines) {
408
409             // lineClipMidPoint(WINMIN, WINMAX, line.p1, line.p2);
410             // lineClipCohSuth(WINMIN, WINMAX, line.p1, line.p2);
411             lineClipLiangBarsk(WINMIN, WINMAX, line.p1, line.p2);
412         }
413     }
414     glutSwapBuffers();
415
416 }
417
418
419 int main(int argc, char *argv[]) {
420
421     /**
422     * first step, init and create window
423     */
424     glutInit(&argc, argv); //init GLUT
425     // init window
426     glutInitWindowPosition(250, 100);

```

```
427     glutInitWindowSize(WIDTH, HEIGHT);
428     // set display mode
429     glutInitDisplayMode(GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGBA);
430     glutCreateWindow("直线裁剪算法"); // create window
431
432     /**
433      * second, register call back functions.
434      */
435     glutDisplayFunc(renderScene);
436     glutKeyboardFunc(processNormalKeys);
437     glutMouseFunc(mouse);
438
439     /**
440      * third, Go into Main loop and listen for events.
441      */
442     // 用白色 刷新屏幕
443     glClearColor(WHITE, 1.0);
444     // 指定绘制颜色为黑色
445     glColor3f(BLACK);
446     glutMainLoop();
447
448     return 1;
449 }
```

实验四 自由曲线绘制

一、实验目的

1. 熟悉 Bezier 曲线生成;
2. 熟悉 B 样条曲线生成;

二、实验内容

- 1、 实现 OpenGL 绘图环境和 Windows 绘图环境兼容;
- 2、 利用 OpenGL 绘制 Bezier 曲线
- 3、 利用 OpenGL 绘制 B 样条 曲线

三、实验环境

PC 微机

Windows 10 操作系统

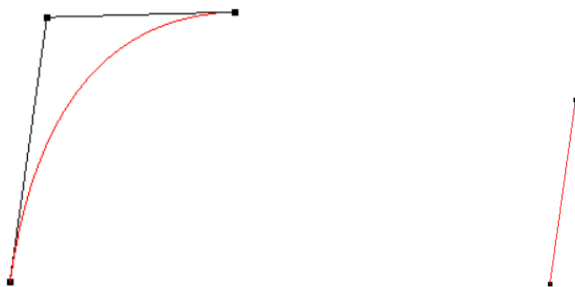
CLion 集成开发环境

四、实验步骤

1. 利用 OpenGL 绘制 Bezier 曲线
2. 利用 OpenGL 绘制 B 样条 曲线

五、实验结果

程序能够与键盘交互，用户鼠标左击确定点击插值的点，键盘敲击 r 可以对之前选择的点位进行修改，敲击 c, 可以重新绘制，敲击 q 可以退出程序。



1. Bezier 曲线

算法伪代码

Input: array $P[0:n]$ of $n+1$ points and real number u in $[0,1]$
Output: point on curve, $C(u)$
Working: point array $Q[0:n]$

```

for  $i := 0$  to  $n$  do
     $Q[i] := P[i]$ ; // save input
for  $k := 1$  to  $n$  do
    for  $i := 0$  to  $n - k$  do
         $Q[i] := (1 - u)Q[i] + u Q[i + 1]$ ;
return  $Q[0]$ ;

```

代码:

```

// 绘制bezier曲线
//
void drawBezier(Point *p) {
    if (numberOfControlPoints <= 0) return;

    Point *p1;
    p1 = new Point[100];

    GLfloat u;
    int i, num = 0;

    for (u=0;u<=1;u=u+(1.0/acc)) {
        auto working : vector<Point> = my_copy(p, numberOfControlPoints);

        for (int k = 1; k <= numberOfControlPoints; k++) {
            for (int j = 0; j < numberOfControlPoints - k; j++) {
                auto temp1 : Point = (working[j] * (1-u));
                auto temp2 : Point = (working[j+1] * u);
                working[j] = temp1 + temp2;
            }
        }

        for (u=0;u<=1;u=u+(1.0/acc)) {
            auto working : vector<Point> = my_copy(p, numberOfControlPoints);

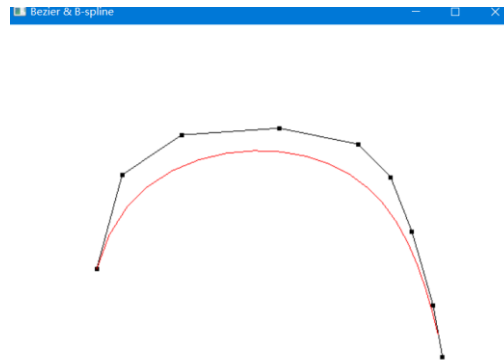
            for (int k = 1; k <= numberOfControlPoints; k++) {
                for (int j = 0; j < numberOfControlPoints - k; j++) {
                    auto temp1 : Point = (working[j] * (1-u));
                    auto temp2 : Point = (working[j+1] * u);
                    working[j] = temp1 + temp2;
                }
            }

            p1[num].x = working[0].x;
            p1[num].y = working[0].y;
            num++;
        }

        for (i = 0; i < numberOfControlPoints; i++) {
            glPointSize( size: 5.0);
            glColor3f( red: 0.0, green: 0.0, blue: 0.0);
            if (i == editIndex)
                glColor3f( red: 1.0, green: 0.0, blue: 0.0);
            setPoint( p: controlPoints[i]);
            if (i != 0)
                setLine( p1: controlPoints[i - 1], p2: controlPoints[i]);
        }
    }
}

```

运行结果:



2. B 样条曲线

算法伪代码:

Input: n, p, m, u , and $m+1$ clamped knots $\{u_0, \dots, u_m\}$
Output: Coefficients $N_{0,p}(u), N_{1,p}(u), \dots, N_{n,p}(u)$ in $N[0], N[1], \dots, N[n]$
Algorithm:
 Initialize $N[0..n]$ to 0; // initialization
 if $u = u_0$ then // rule out special cases
 $N[0] = 1.0$;
 return
 else $u = u_m$ then
 $N[n] = 1.0$
 return
 end if
 // now u is between u_0 and u_m
 Let u be in knot span $[u_k, u_{k+1})$;
 $N[k] := 1.0$ // degree 0 coefficient
 for $d := 1$ to p do // degree d goes from 1 to p
 begin
 $N[k-d] = \frac{u_{k+1} - u}{u_{k+1} - u_{(k-d)+1}} * N[(k-d)+1]$; // right (south-west corner) term only
 for $i := k-d+1$ to $k-1$ do // compute internal terms
 $N[i] := \frac{u - u_i}{u_{i+d} - u_i} * N[i] + \frac{u_{i+d+1} - u}{u_{i+d+1} - u_{i+1}} * N[i+1]$;
 $N[k] = \frac{u - u_k}{u_{k+d} - u_k} * N[k]$; // left (north-west corner) term only
 end
 // array $N[0..n]$ has the coefficients.

代码:

```
void drawBSpline(Point *p) {
    if (numberOfControlPoints <= 0) return;
    Point *p1;
    p1 = new Point[100];
    GLfloat u = 0, x, y;
    int i, num = 0;

    for (u = 0; u <= 1; u = u + (1.0 / acc)) {
        x = 0;
        y = 0;
        calN(t, u);
        // 曲线次数=控制点个数-1
        for (i = 0; i <= numberOfControlPoints - 1; i++) {
            x += N[i] * p[i].x;
            y += N[i] * p[i].y;
        }
        p1[num].x = x;
        p1[num].y = y;
        num++;
    }
}
```

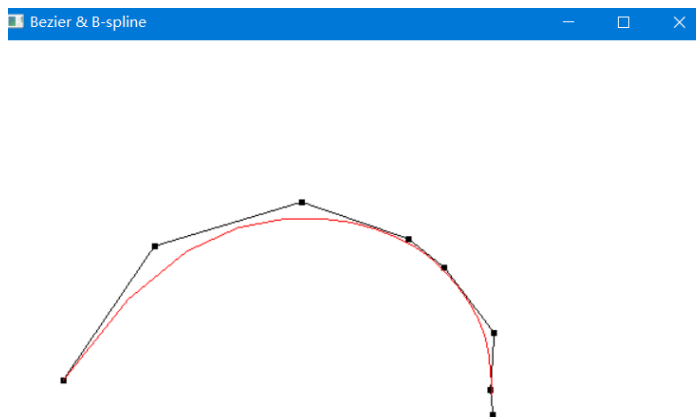
```

for (i = 0; i < numberOfControlPoints; i++) {
    glPointSize( size: 5.0);
    glColor3f( red: 0.0, green: 0.0, blue: 0.0);
    if (i == editIndex)
        glColor3f( red: 1.0, green: 0.0, blue: 0.0);
    //      setPoint(controlPoints[i]);
    if (i != 0)
        setLine( p1: controlPoints[i - 1], p2: controlPoints[i]);
}

glColor3f( red: 1.0, green: 0.0, blue: 0.0);
glBegin( mode: GL_LINE_STRIP);
for (int k = 0; k < num; k++)
    glVertex2f( x: p1[k].x, y: p1[k].y);
glEnd();
delete[] p1;
glFlush();

```

结果：



六、实验总结

1. Bezier 曲线绘制原理

Bezier 曲线是一种参数曲线，能够绘制任意 2 维图形。

它的方程式为：

$$C(u) = \sum_{i=0}^n B_{n,i}(u)p_i$$

$$B_{n,i}(u) = \frac{n!}{i! \times (n-i)!} u^i (1-u)^{n-i}$$

我们以 4 阶贝塞尔曲线的系数为例：

$$(u + (1-u))^n \Rightarrow \text{展开式}$$

$$\begin{aligned} B_{4,0} &= \binom{4}{0} u^0 (1-u)^4 & B_{4,3} &= \binom{4}{3} u^3 (1-u) \\ B_{4,1} &= \binom{4}{1} u^1 (1-u)^3 & B_{4,4} &= \binom{4}{4} u^4 (1-u)^0 \\ B_{4,2} &= \binom{4}{2} u^2 (1-u)^2 \end{aligned}$$

要绘制曲线，也就是在 $[0, 1]$ 上取一些点，然后根据公式计算出我们所取点所对应的曲线上的点，再将点连接起来，就是我们需要的曲线。

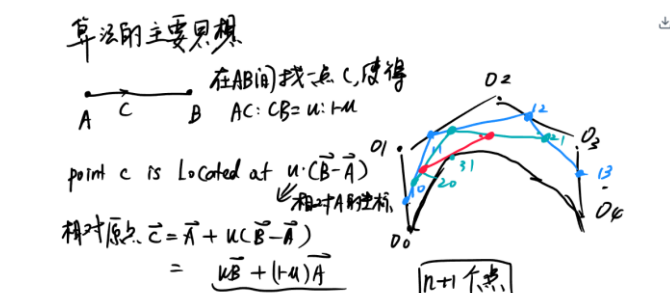
```
// 使用C++实现B-Spline曲线
for(i = 0; i <= num_cp-1; i++)
{
    x+=comb(num_cp-1,i)*pow(u,i)*pow((1-u),(num_cp-1-i))*p[i].x;
    y+=comb(num_cp-1,i)*pow(u,i)*pow((1-u),(num_cp-1-i))*p[i].y;
}
p1[num].x=x;
p1[num].y=y;
num++;
}
```

我们可以选择直接将系数计算出来后，再将 u 值代入公式计算对应点，但是这样总是需要计算阶乘，非常耗时，我们可以换一种思路。

首先我们将点进行编号

points	iter	Label
P_0	0	0
P_1	0	1
P_2	0	2
$\vdots$	$\vdots$	$\vdots$
P_n	0	n

再来介绍算法的思想：



2. B-Spline 曲线

首先，B 样条曲线是一种参数曲线，即将 $[0,1]$ 范围内（0 代表曲线起点,1 代表终点）的一个参数 u 的值映射到二维（或更高维，不过本文主要讨论二维）空间上。B-样条曲线包含以下属性

节点(knots): 给定集合 U ，它包含 $m + 1$ 个有理数 $u_0, u_1, u_2, \dots, u_m$ ，且满足：

$$u_0 \leq u_1 \leq u_2 \leq \dots \leq u_m$$

节点矢量(knot vector): 由节点数据组成的 $n + 1$ 维矢量 $[u_0, u_1, u_2, \dots, u_m]$ 。

节点区间(knot span): $[u_i, u_{i+1})$ 称为第 i 个节点区间。

k重节点(k-multiple knot): 如果 $u_i = u_{i+1} = \dots = u_{i+k-1}$ 且 $k > 1$ ，则称 u_i 为 k 重节点。

基函数(Basis functions)/系数(Coefficients)/权重(Weights): 可以看作控制点对曲线上的点的作用力，且曲线上任何一点受到的来自多个控制点的作用权重之和为1。

B-Spline 曲线的计算公式为

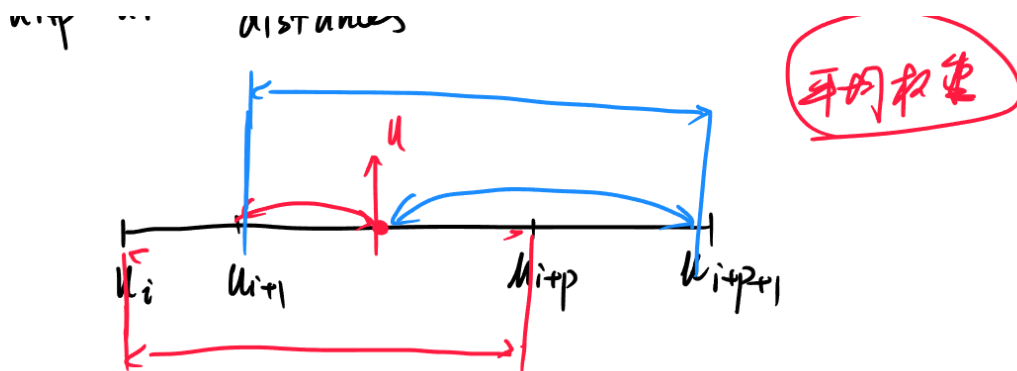
$$\mathbf{C}(u) = \sum_{i=1}^n N_{i,p}(u) \mathbf{P}_i$$

$$N_{i,0}(u) = \begin{cases} 1 & \text{if } u_i \leq u < u_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u)$$

可以看出 B 样条曲线和贝塞尔曲线的计算公式相当类似，都是系数乘上坐标再求和，但是值得注意的是，B 样条曲线的阶是作为输入，而不是作为输出。

做实验尤其重要的是理解公式的含义，这个递推式是怎么来的呢？下面这张图给出了很好的解释。



在计算 $N_{i,p}(u)$ 时，它使用了 $N_{i,p-1}(u)$ 和 $N_{i+1,p-1}(u)$ 。前者仅在 $[u_i, u_{i+p})$ 存在。如果 u 在此半开区间内，则 $u - u_i$ 是 u 与该区间左端之间的距离，该区间长度为 $u_{i+p} - u_i$ ，而 $(u - u_i) / (u_{i+p} - u_i)$ 是上述距离的比例，始终在 0 到 1 的范围内。递推式的另一项同理可得。

3. 在实验中，我使用了 c++ 的运算符重载，对 Point 类的加，乘进行重载

```

class Point{
public:
    GLfloat x, y;

    Point operator + (Point& other) const {
        return { .x: this->x + other.x, .y: this->y + other.y };
    }

    Point operator * (float factor) const {
        return { .x: this->x * factor, .y: this->y * factor };
    }

    friend std::ostream& operator << (std::ostream& out, Point& p) {
        out << "(" << p.x << ", " << p.y << ")" << std::endl;
        return out;
    }
};

```

4. 实现撤销功能总结

我们可以通过键盘回调函数 `glutKeyboardFunc`, 注册一个函数, 让程序在用户按 `r` 时, 修改全局变量 `numberOfCurrentPoints`.

```

void keyboard(unsigned char key, int x, int y) {
    switch (key) {
        //Q 键退出程序
        case 'q':
        case 'Q':
            exit( Code: 0);
            break;
        //C 键重绘
        case 'c':
        case 'C':
            numberOfControlPoints = 0;
            glutPostRedisplay();
            break;
        //R 键撤销一个点
        case 'r':
        case 'R':
            numberOfControlPoints--;
            glutPostRedisplay();
            break;
    }
}

```

5. 实验总结

首先, 我们对 Bezier 曲线进行了研究。Bezier 曲线是一种二维图形, 其定义为由多个控制点连接起来的曲线。通过使用 OpenGL 的函数, 我们可以计算出 Bezier 曲线上每个点的坐标, 并将其绘制出来。在绘制过程中, 我们需要注意曲线的光滑度以及控制点的位置。

接下来, 我们研究了 B 样条曲线。B 样条曲线是一种特殊的曲线, 其具有比较高的光滑度和更好的外观效果。它的实现与 Bezier 曲线相似, 但需要对控制点进行额外的处理。我们需要根据控制点的位置计算出 B 样条曲线上的每个点的坐标, 并绘制出曲线。

总的来说, 通过本次实验, 我们对曲线的概念以及使用 OpenGL 绘制图形的方法有了更深入的了解。我们学会了如何使用 OpenGL 绘制 Bezier 曲线和 B 样条曲线, 并对这两种曲线的特点进行了对比。

七、实验代码

```

1 #include <GL/GLUT.h>
2 #include <cmath>
3 #include <iostream>
4 #include <vector>
5
6 #define MAX_POINTS 9 //控制点数量=MAX_CP+1
7 #define MAX_P 5 //支持的最高幂次
8
9
10 int numberOfControlPoints = 0; //控制点个数 =n+1
11 int p = 3; //幂次
12 int m; //实际结点个数-1
13 float N[MAX_POINTS + 1]; //最大系数数组
14 float u[MAX_POINTS + 1 + MAX_P + 1]; //m+1 个节点组成的向量数组
15 int width = 600, height = 600;
16
17 // 初始化基函数表
18 //
19 void initN()
20 {
21     for(int i = 0; i < numberOfControlPoints; i++)
22         N[i] = 0;
23 }
24
25 // 根据节点个数初始化节点表, 默认为均匀节点
26 //
27 void initU()
28 {
29     m = numberOfControlPoints + p;
30     for(int i = 0; i < m+1; i++)
31     {
32         if(i <= p)
33         {
34             u[i] = 0;
35             continue;
36         }
37         else if(i >= m-p)
38         {
39             u[i] = 1;
40             continue;
41         }
42         u[i] = (float) (i-p) / (float) (m-2*p);
43     }
44 }

```

```

45
46 // 根据曲线参数 t 计算基函数序列
47 //
48 void calN(float t)
49 {
50     //将数组 N 初始化
51     initN();
52     initU();
53     //处理边界情况
54     if(0 == t)
55     {
56         N[0] = 1.0;
57         return;
58     }
59     else if(1 == t)
60     {
61         N[numberOfControlPoints - 1] = 1.0;
62         return;
63     }
64     //遍历节点表, 找出 t 所在的区间: u[k] <= t < u[k+1]
65     int i, k = 0;
66     for(i = 1; i < m+1; i++)
67         if(t < u[i])
68         {
69             k = i-1;
70             break;
71         }
72     N[k] = 1.0;
73     //阶数 d
74     for(int d = 1; d <= p; d++)
75     {
76         N[k-d] = N[k-d+1] * (u[k+1]-t) / (u[k+1]-u[k-d+1]);
77         for(i = k-d+1; i <= k-1; i++)
78             N[i] = N[i] * (t-u[i]) / (u[i+d]-u[i]) + N[i+1] *
(u[i+d+1]-t) / (u[i+d+1]-u[i+1]);
79         N[k] = N[k] * (t-u[k]) / (u[k+d]-u[k]);
80     }
81 }
82
83
84
85 class Point{
86 public:
87     GLfloat x, y;

```

```

88
89     Point operator + (Point& other) const {
90         return {this->x + other.x, this->y + other.y};
91     }
92
93     Point operator * (float factor) const {
94         return {this->x * factor, this->y * factor };
95     }
96
97     friend std::ostream& operator << (std::ostream& out, Point& p)
98 {
99     out << "(" << p.x << ", " << p.y << ")" << std::endl;
100     return out;
101 } ;
102
103 // 存储控制点的转换坐标
104
105 Point controlPoints[MAX_POINTS];
106 // 存储控制点的实际坐标
107
108 Point Rcpts[MAX_POINTS];
109
110 bool editFlag = false;
111 int editIndex = -1;
112
113 // 精度(Accuracy)
114 //
115 int acc = 20;
116
117
118 void setPoint(Point p) {
119
120     using namespace std;
121     cout << p << endl;
122     glBegin(GL_POINTS);
123     glVertex2f(p.x, p.y);
124     glEnd();
125     glFlush();
126 }
127
128
129 void setLine(Point p1, Point p2) {
130     glColor3f(0.0, 0.0, 0.0);

```

```

131     glBegin(GL_LINES);
132     glVertex2f(p1.x, p1.y);
133     glVertex2f(p2.x, p2.y);
134     glEnd();
135     glFlush();
136 }
137
138
139 std::vector<Point> my_copy( const Point* p, int
numberOfControlPoints) {
140     std::vector<Point> res;
141     for (int i = 0; i < numberOfControlPoints; i++) {
142         res.push_back(p[i]);
143     }
144     return res;
145 }
146
147 // 绘制 bezier 曲线
148 //
149 void drawBezier(Point *p) {
150     if (numberOfControlPoints <= 0) return;
151
152     Point *p1;
153     p1 = new Point[100];
154
155     GLfloat u ;
156     int i, num = 0;
157
158
159     for (u=0;u<=1;u=u+(1.0/acc)) {
160         auto working = my_copy(p, numberOfControlPoints);
161
162         for (int k = 1; k <= numberOfControlPoints; k++) {
163             for (int j = 0; j < numberOfControlPoints - k; j++) {
164                 auto temp1 =(working[j] * (1-u));
165                 auto temp2 =(working[1+j] * u) ;
166                 working[j] = temp1 + temp2;
167             }
168         }
169         p1[num].x = working[0].x;
170         p1[num].y = working[0].y;
171         num++;
172     }
173

```

```

174
175     for (i = 0; i < numberOfControlPoints; i++) {
176         glPointSize(5.0);
177         glColor3f(0.0, 0.0, 0.0);
178         if (i == editIndex)
179             glColor3f(1.0, 0.0, 0.0);
180         setPoint(controlPoints[i]);
181         if (i != 0)
182             setLine(controlPoints[i - 1], controlPoints[i]);
183     }
184
185     glColor3f(1.0, 0.0, 0.0);
186     glBegin(GL_LINE_STRIP);
187     for (int k = 0; k < num; k++)
188         glVertex2f(p1[k].x, p1[k].y);
189     glEnd();
190     delete[] p1;
191     glFlush();
192 }
193
194 void drawBSpline(Point *p) {
195     if (numberOfControlPoints <= 0) return;
196     Point *p1;
197     p1 = new Point[100];
198     GLfloat u = 0, x, y;
199     int i, num = 0;
200
201     for (u = 0; u <= 1; u = u + (1.0 / acc)) {
202         x = 0;
203         y = 0;
204         calN(u);
205         //曲线次数=控制点个数-1
206         for (i = 0; i <= numberOfControlPoints - 1; i++) {
207             x += N[i] * p[i].x;
208             y += N[i] * p[i].y;
209         }
210         p1[num].x = x;
211         p1[num].y = y;
212         num++;
213     }
214
215
216     for (i = 0; i < numberOfControlPoints; i++) {
217         glPointSize(5.0);

```



```

218     glColor3f(0.0, 0.0, 0.0);
219     if (i == editIndex)
220         glColor3f(1.0, 0.0, 0.0);
221 //     setPoint(controlPoints[i]);
222     if (i != 0)
223         setLine(controlPoints[i - 1], controlPoints[i]);
224 }
225
226 glColor3f(1.0, 0.0, 0.0);
227 glBegin(GL_LINE_STRIP);
228 for (int k = 0; k < num; k++)
229     glVertex2f(p1[k].x, p1[k].y);
230 glEnd();
231 delete[] p1;
232 glFlush();
233 }
234
235 void display(void) {
236     glClear(GL_COLOR_BUFFER_BIT);
237     glFlush();
238 //     setPoint(Point{0.0784641, 0.60601});
239 //     drawBSpline(controlPoints);
240     drawBezier(controlPoints);
241 }
242
243
244 static void mouse(int button, int state, int x, int y) {
245     float wx, wy;
246     if (state == GLUT_DOWN) {
247         //鼠标左键为绘制
248
249         if (button == GLUT_LEFT_BUTTON) {
250             //判断控制点数目是否超过最大值
251             if (numberOfControlPoints == MAX_POINTS)
252                 return;
253             Rcpts[numberOfControlPoints].x = x;
254             Rcpts[numberOfControlPoints].y = y;
255             //转换坐标
256             wx = (2.0 * x) / (float) (width - 1) - 1.0;
257             wy = (2.0 * (height - 1 - y)) / (float) (height - 1) -
1.0;
258             //存储控制点
259             controlPoints[numberOfControlPoints].x = wx;
260             controlPoints[numberOfControlPoints].y = wy;

```

```

261         numberOfControlPoints++;
262         display();
263     }
264     //非编辑状态下, 右键点选后, 将待修改的点进行标记
265     if (button == GLUT_RIGHT_BUTTON && editFlag == false) {
266         for (int i = 0; i < numberOfControlPoints; i++) {
267             if (pow(x - Rcpts[i].x, 2) + pow(y - Rcpts[i].y, 2)
< 100.0) {
268                 editIndex = i;
269                 editFlag = true;
270                 display();
271                 drawBezier(controlPoints);
272                 return;
273             }
274         }
275     }
276     //已经有标记点的情况下, 再按右键, 记录这个点的最终位置
277     if (button == GLUT_RIGHT_BUTTON && editFlag == true) {
278         wx = (2.0 * x) / (float) (width - 1) - 1.0;
279         wy = (2.0 * (height - 1 - y)) / (float) (height - 1) -
1.0;
280         Rcpts[editIndex].x = x;
281         Rcpts[editIndex].y = y;
282         controlPoints[editIndex].x = wx;
283         controlPoints[editIndex].y = wy;
284         editFlag = false;
285         display();
286     }
287 }
288 }
289
290 // 键盘回调函数
291 //
292 void keyboard(unsigned char key, int x, int y) {
293     switch (key) {
294         //Q 键退出程序
295         case 'q':
296         case 'Q':
297             exit(0);
298             break;
299         //C 键重绘
300         case 'c':
301         case 'C':
302             numberOfControlPoints = 0;

```

```

303         glutPostRedisplay();
304         break;
305         //R 键撤销一个点
306         case 'r':
307         case 'R':
308             numberOfControlPoints--;
309             glutPostRedisplay();
310             break;
311     }
312 }
313
314
315 void keyboard1(int key, int x, int y) {
316     switch (key) {
317         case GLUT_KEY_UP:
318             if (acc < 100)
319                 acc += 1;
320             glutPostRedisplay();
321             break;
322         case GLUT_KEY_DOWN:
323             if (acc >= 5)
324                 acc -= 1;
325             glutPostRedisplay();
326             break;
327     }
328 }
329
330 //void reshape(int w, int h) {
331 //    glMatrixMode(GL_PROJECTION);
332 //    glLoadIdentity();
333 //    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
334 //    glMatrixMode(GL_MODELVIEW);
335 //    glViewport(0, 0, w, h); //调整视口
336 //    width = w;
337 //    height = h;
338 //}
339
340 int main(int argc, char **argv) {
341     //初始化
342     glutInit(&argc, argv);
343     glutInitDisplayMode(GLUT_RGB);
344     glutInitWindowSize(width, height);
345     glutCreateWindow("DrawBezier");
346     //注册回调函数

```

```
347     glutDisplayFunc (display);
348     glutMouseFunc (mouse);
349     glutKeyboardFunc (keyboard);
350     glutSpecialFunc (keyboard1);
351     //     glutReshapeFunc (reshape);
352     glClearColor(1.0, 1.0, 1.0, 1.0);
353     glColor3f(0.0, 0.0, 0.0);
354     glutMainLoop();
355 }
```

实验五 OpenGL 综合实验

一、实验目的

- 1、配置 OpenGL 环境
- 2、熟悉 OpenGL 的主要功能；
- 2、掌握 OpenGL 的绘图流程和原理；

二、实验内容

- 1、 实现 OpenGL 绘图环境和 Windows 绘图环境兼容；
- 2、 综合本学期内容，使用 openGL 写出一个比较综合的项目

三、实验环境

PC 微机

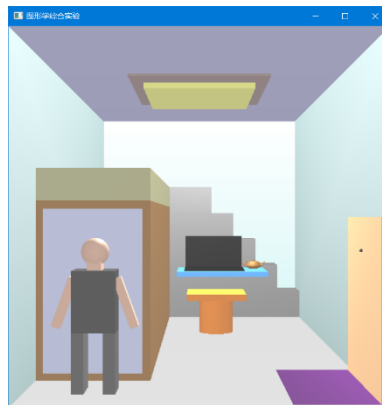
Windows 10 操作系统

CLion 集成开发环境

四、实验结果

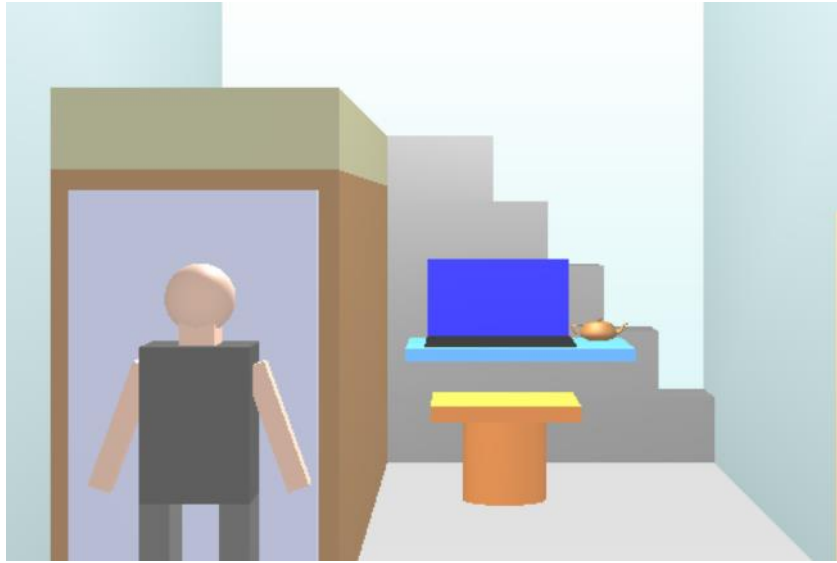
本项目利用 OpenGL 制作了一个简易的房间内饰，并且可以实现简单交互

项目概览



本项目可以结合键盘实现九大功能。

1. 关闭电脑



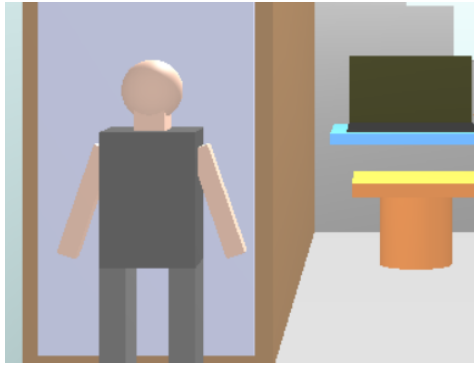
2. 打开电脑



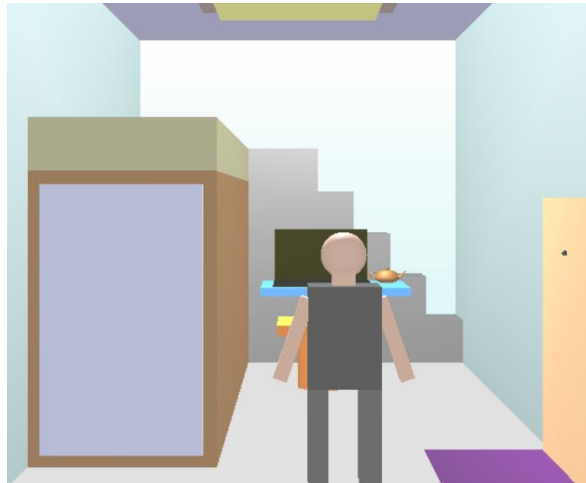
3. 人物举手



4. 人物放下手

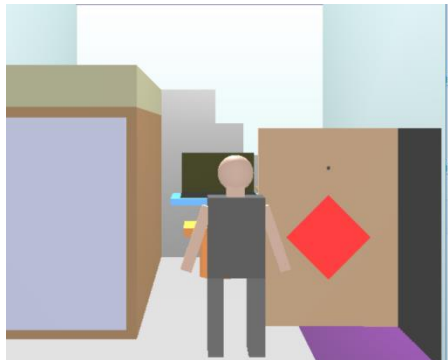


5. 人物向右走

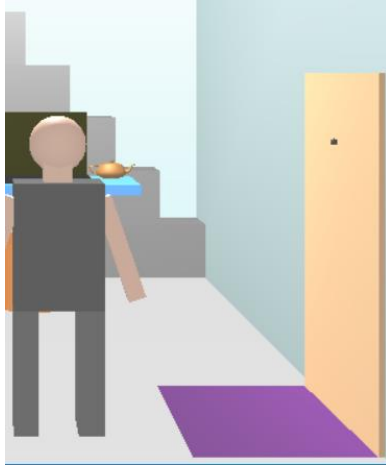


6. 人物向左走

7. 开门



8. 关门



9. 关灯



五、实验总结

本 OpenGL 程序的结构大致分为五部分。

结构介绍

1. 初始化 OpenGL 和 创建窗口

```
/**
 * first step, init and create window
 */
glutInit( argc, &argv ); //init GLUT
// init window
glutInitWindowPosition( x: 250, y: 100 );
glutInitWindowSize( width: 600, height: 600 );
// set display mode
glutInitDisplayMode( displayMode: GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGBA );
glutCreateWindow( title: "tutors" ); // create window
```


2. 显示回调函数

`glutDisplayFunc( callback: display);`

实现回调函数

```
void display(void) // Here's Where We Do All The Drawing
{
    glPushMatrix();
    //glClearColor(1.0, 1.0, 1.0, 0.0);
    glClear( mask: GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT); // 必须放在display中否则动画会有bug
    // TODO:
    // Place light source here
    GLfloat lightpos[] = { [0]: 0.f, [1]: 100.f, [2]: -450.f, [3]: 1.f }; // 可设动画
    GLfloat light0_mat1[] = { [0]: 0.2, [1]: 0.2, [2]: 0.2, [3]: 0.3f };
    GLfloat light0_diff[] = { [0]: 1.0, [1]: 1.0, [2]: 1.0, [3]: 0.3 };
    GLfloat lightpos2[] = { [0]: 0.0, [1]: 0.0, [2]: 0.0, [3]: 1.f };
    GLfloat light1_mat1[] = { [0]: 0.1, [1]: 0.1, [2]: 0.1, [3]: 0.3 };
    GLfloat light1_diff[] = { [0]: 0.1, [1]: 0.1, [2]: 0.1, [3]: 0.3 };
    glLightfv( light: GL_LIGHT0, pname: GL_POSITION, params: lightpos);
    glLightfv( light: GL_LIGHT0, pname: GL_AMBIENT, params: light0_mat1);
    glLightfv( light: GL_LIGHT0, pname: GL_DIFFUSE, params: light0_diff);
    glLightfv( light: GL_LIGHT1, pname: GL_POSITION, params: lightpos2);
    glLightfv( light: GL_LIGHT1, pname: GL_AMBIENT, params: light1_mat1);
    glLightfv( light: GL_LIGHT1, pname: GL_DIFFUSE, params: light1_diff);

    GLfloat no_mat[] = { [0]: 0.0, [1]: 0.0, [2]: 0.0, [3]: 1.0 };
    GLfloat mat_ambient[] = { [0]: 0.2, [1]: 0.2, [2]: 0.2, [3]: 1.0 };
    GLfloat mat_diffuse[] = { [0]: 0.9, [1]: 0.9, [2]: 0.9, [3]: 1.0 };
    GLfloat mat_specular[] = { [0]: 0.3, [1]: 0.3, [2]: 0.3, [3]: 1.0 };
```

3. 设置改变大小时的回调函数

```
void reshape(int w, int h) // Resize the GL Window. w=width, h=height
{
    glViewport( x: 0, y: 0, width: (GLsizei)w, height: (GLsizei)h);
    glMatrixMode( mode: GL_PROJECTION);
    glLoadIdentity();
    glFrustum( left: -FRUSTDIM, right: FRUSTDIM, bottom: -FRUSTDIM, top: FRUSTDIM, zNear: 300., zFar: 800.);
    glMatrixMode( mode: GL_MODELVIEW);
    glLoadIdentity();
}
```

4. 设置键盘回调函数

`glutKeyboardFunc( callback: keyboard);`

```

void keyboard(unsigned char key, int x, int y) // Handle the keyboard events here
{
    switch (key)
    {
        case '\033': // press 'esc' to quit
            exit( Code: 0);
            break;

        // TODO:
        // Add keyboard control here
        case '1':
            printf( format: "computer power on\n");
            option = 1;
            break;
        case '2':
            printf( format: "computer power off\n");
            option = 2;
            break;
        ...
    }
}

```

5.显示动画, idle 回调函数

```

glutIdleFunc( callback: idle);

```

```

void idle()
{
    if (option == 1) {
        if (screen <= 1.0) {
            Sleep( dwMilliseconds: 100);
            screen += 0.1;
        }
    }
    if (option == 2) {
        if (screen >= 0.0) {
            Sleep( dwMilliseconds: 100);
            screen -= 0.1;
        }
    }
    if (option == 3) {
        if (arm <= 130.0) {
            Sleep( dwMilliseconds: 5);
            arm += 1;
        }
    }
    if (option == 4) {

```

程序的主要实现难点是陈列物品的 display 函数，以及显示动画的 idle 函数。以下将详细介绍：

Display 函数

1. 首先定义常用数组：

```
// Place light source here
GLfloat lightpos[] = { [0]: 0.f, [1]: 100.f, [2]: -450.f, [3]: 1.f }; // 可设动画
GLfloat light0_mat1[] = { [0]: 0.2, [1]: 0.2, [2]: 0.2, [3]: 0.3f };
GLfloat light0_diff[] = { [0]: 1.0, [1]: 1.0, [2]: 1.0, [3]: 0.3 };
GLfloat lightpos2[] = { [0]: 0.0, [1]: 0.0, [2]: 0.0, [3]: 1.f };
GLfloat light1_mat1[] = { [0]: 0.1, [1]: 0.1, [2]: 0.1, [3]: 0.3 };
GLfloat light1_diff[] = { [0]: 0.1, [1]: 0.1, [2]: 0.1, [3]: 0.3 };
glLightfv( light: GL_LIGHT0, pname: GL_POSITION, params: lightpos);
glLightfv( light: GL_LIGHT0, pname: GL_AMBIENT, params: light0_mat1);
glLightfv( light: GL_LIGHT0, pname: GL_DIFFUSE, params: light0_diff);
glLightfv( light: GL_LIGHT1, pname: GL_POSITION, params: lightpos2);
glLightfv( light: GL_LIGHT1, pname: GL_AMBIENT, params: light1_mat1);
glLightfv( light: GL_LIGHT1, pname: GL_DIFFUSE, params: light1_diff);

GLfloat no_mat[] = { [0]: 0.0, [1]: 0.0, [2]: 0.0, [3]: 1.0 };
GLfloat mat_ambient[] = { [0]: 0.2, [1]: 0.2, [2]: 0.2, [3]: 1.0 };
GLfloat mat_diffuse[] = { [0]: 0.9, [1]: 0.9, [2]: 0.9, [3]: 1.0 };
GLfloat mat_specular[] = { [0]: 0.3, [1]: 0.3, [2]: 0.3, [3]: 1.0 };
GLfloat high_shininess[] = { [0]: 20.0 };
GLfloat high_mat[] = { [0]: 1.f, [1]: 1.f, [2]: 1.f, [3]: 1.0 };

glMaterialfv( face: GL_FRONT, pname: GL_AMBIENT, params: mat_ambient);
glMaterialfv( face: GL_FRONT, pname: GL_DIFFUSE, params: mat_diffuse);
glMaterialfv( face: GL_FRONT, pname: GL_SPECULAR, params: mat_specular);
glMaterialfv( face: GL_FRONT, pname: GL_SHININESS, params: high_shininess);
glMaterialfv( face: GL_FRONT, pname: GL_EMISSION, params: high_mat);
```

这段 OpenGL 代码的功能是在 OpenGL 3D 场景中设置光源和物体的材质属性。

它首先定义了两个光源的位置：lightpos 和 lightpos2。然后，使用 glLightfv 函数设置了每个光源的位置，环境光和散射光。

接下来，使用 glMaterialfv 函数设置了物体的材质属性，包括环境光、散射光、镜面光、高光指数和发射光。

最后，所有的光源和材质属性都将用于在 3D 场景中渲染物体，以便产生阴影和光照效果。

数组名称	含义
lightpos	设置第一个光源的位置，最后一个数字可以设为 1，以使光源变成平行光源
light0_mat1	设置第一个光源的环境光参数，用于控制物体对全局光照的反应
light0_diff	设置第一个光源的漫反射光参数，用于控制物体对漫反射光的反应
lightpos2	设置第二个光源的位置，最后一个数字可以设为 1，以使光源变成平行光源
light1_mat1	设置第二个光源的环境光参数，用于控制物体对全局光照的反应
light1_diff	设置第二个光源的漫反射光参数，用于控制物体对漫反射光的反应
no_mat	用于设置物体不发射任何光
mat_ambient	用于设置物体的环境光参数，控制物体对全局光照的反应
mat_diffuse	用于设置物体的漫反射光参数，控制物体对漫反射光的反应
mat_specular	用于设置物体的镜面反射光参数，控制物体对镜面反射光的反应
high_shininess	用于设置物体的镜面反射光的亮度

2. 使用 QUADS 类型绘制上下左右方的墙壁

```
// back wall
glColor3f( red: 0.5, green: 0.6, blue: 0.6);
glBegin( mode: GL_QUADS);
glNormal3f( nx: 0.0, ny: 0.0, nz: 1.0);
glVertex3f( x: 100.0, y: 100.0, z: -300.0);
glVertex3f( x: -100.0, y: 100.0, z: -300.0);
glVertex3f( x: -100.0, y: -100.0, z: -300.0);
glVertex3f( x: 100.0, y: -100.0, z: -300.0);
glEnd();

// left wall
glColor3f( red: 0.5, green: 0.6, blue: 0.6);
glBegin( mode: GL_QUADS);
glNormal3f( nx: 1.0, ny: 0.0, nz: 0.0);
glVertex3f( x: -100.0, y: -100.0, z: 0.0);
glVertex3f( x: -100.0, y: -100.0, z: -300.0);
glVertex3f( x: -100.0, y: 100.0, z: -300.0);
glVertex3f( x: -100.0, y: 100.0, z: 0.0);
glEnd();

// right wall
glColor3f( red: 0.5, green: 0.6, blue: 0.6);
glBegin( mode: GL_QUADS);
glNormal3f( nx: -1.0, ny: 0.0, nz: 0.0);
glVertex3f( x: 100.0, y: -100.0, z: 0.0);
glVertex3f( x: 100.0, y: 100.0, z: 0.0);
glVertex3f( x: 100.0, y: 100.0, z: -300.0);
glVertex3f( x: 100.0, y: -100.0, z: -300.0);
glEnd();
```

3. 使用立方体生成桌子和镜子

```
// bed
glPushMatrix();
glColor3f( red: 0.7, green: 0.7, blue: 0.5);
glTranslatef( x: -65.0, y: 20.0, z: -175.0);
glScalef( x: 3.5, y: 1.0, z: 12.5);
glutSolidCube( size: 20.0);
glPopMatrix();

// wardrobe
glPushMatrix();
glColor3f( red: 0.6, green: 0.4, blue: 0.2);
glTranslatef( x: -65.0, y: -45.0, z: -175.0);
glScalef( x: 14.0, y: 22.0, z: 50.0);
glutSolidCube( size: 5.0);
glPopMatrix();

// mirror
glPushMatrix();
glTranslatef( x: -65.0, y: -45.0, z: -49.0);
glColor3f( red: 0.79, green: 0.82, blue: 0.98);
glScalef( x: 60.0, y: 100.0, z: 2.0);
glutSolidCube( size: 1.0);
glPopMatrix();
```



床：使用 `glPushMatrix` 与 `glPopMatrix` 来管理堆栈，先推入矩阵，然后修改矩阵，最后再把矩阵弹出。使用 `glTranslatef` 函数将床移动到(-65.0, 20.0, -175.0)的位置，使用 `glScalef` 函数将床的尺寸调整为(3.5, 1.0, 12.5)，最后使用 `glutSolidCube` 函数绘制一个长 20.0 的立方体。

衣柜：同样使用 `glPushMatrix` 与 `glPopMatrix` 管理堆栈，将衣柜移动到(-65.0, -45.0, -175.0)的位置，调整衣柜的尺寸为(14.0, 22.0, 50.0)，最后绘制一个长 5.0 的立方体。

镜子：同样使用 `glPushMatrix` 与 `glPopMatrix` 管理堆栈，将镜子移动到(-65.0, -45.0, -49.0)的位置，调整镜子的尺寸为(60.0, 100.0, 2.0)，最后绘制一个长 1.0 的立方体。

4. 人物建模

Xiaoming 主要由四个部分组成，即头、身体、手臂、腿组成。分别采用的图形结构是

4.1 首先画了手臂（两个立方体）

```
// arms
glColor3f( red: 0.9, green: 0.7, blue: 0.6);
glPushMatrix();
    glTranslatef( x: -12.5, y: -25.0, z: 0.0);
    glRotatef( angle: arm, x: 0.0, y: 0.0, z: -1.0); // 可设动画
    glTranslatef( x: 0.0, y: -15.0, z: 0.0);
    glScalef( x: 5.0, y: 30.0, z: 5.0);
    glutSolidCube( size: 1.0);
glPopMatrix();
glPushMatrix();
    glTranslatef( x: 12.5, y: -25.0, z: 0.0);
    glRotatef( angle: arm, x: 0.0, y: 0.0, z: 1.0);
    glTranslatef( x: 0.0, y: -15.0, z: 0.0);
    glScalef( x: 5.0, y: 30.0, z: 5.0);
    glutSolidCube( size: 1.0);
glPopMatrix();
```

4.2 绘制头和颈部（球和立方体）

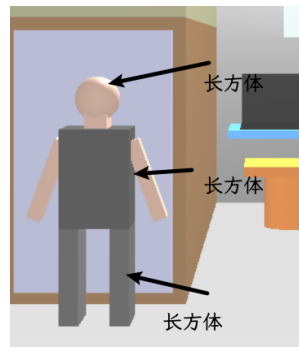
```
// head
glTranslatef( x: 0.0, y: -10.0, z: 0.0);
glColor3f( red: 0.9, green: 0.7, blue: 0.6);
glutSolidSphere( radius: 8.0, slices: 50, stacks: 50);
// neck
glTranslatef( x: 0.0, y: -10.0, z: 0.0);
glScalef( x: 8.0, y: 10.0, z: 10.0);
glutSolidCube( size: 1.0);
```

4.3 绘制身体和腿（立方体）

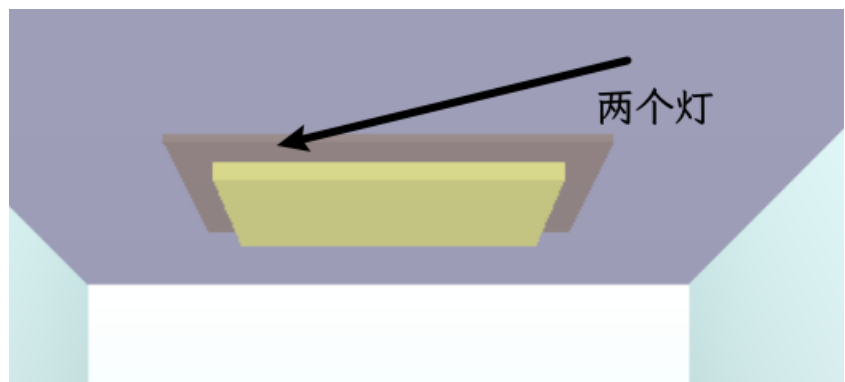
```

// body
glColor3f( red: 0.2, green: 0.2, blue: 0.2);
glScalef( x: 0.125, y: 0.1, z: 0.1);
glTranslatef( x: 0.0, y: -18.0, z: 0.0);
glScalef( x: 25.0, y: 35.0, z: 15.0);
glutSolidCube( size: 1.0);
// legs
glColor3f( red: 0.3, green: 0.3, blue: 0.3);
glScalef( x: 1.0 / 25.0, y: 1.0 / 35.0, z: 1.0 / 15.0);
glTranslatef( x: -9.0, y: -35.0, z: 0.0);
glScalef( x: 7.0, y: 35.0, z: 15.0);
glutSolidCube( size: 1.0);
glScalef( x: 1.0 / 7.0, y: 1.0 / 35.0, z: 1.0 / 15.0);
glTranslatef( x: 18.0, y: 0.0, z: 0.0);
glScalef( x: 7.0, y: 35.0, z: 15.0);
glutSolidCube( size: 1.0);
glPopMatrix();

```



5. 绘制灯 两个长方体



```
// top light
glPushMatrix();
    glColor3f( red: 1.0, green: 1.0, blue: 0.5);
    glTranslatef( x: 0.0, y: 94.0, z: -150.0);
    glScalef( x: 80.0, y: 4.0, z: 80.0);
    glutSolidCube( size: 1.0);
glPopMatrix();
glPushMatrix();
    glColor3f( red: 0.6, green: 0.5, blue: 0.5);
    glTranslatef( x: 0.0, y: 99.0, z: -150.0);
    glScalef( x: 100.0, y: 2.0, z: 100.0);
    glutSolidCube( size: 1.0);
glPopMatrix();
```

上面的代码是实现渲染顶部灯光的代码。使用两个 `glPushMatrix()` 和 `glPopMatrix()` 组合构建了两个立方体，分别表示顶部灯光的底座和灯罩。在每个 `glPushMatrix()` 和 `glPopMatrix()` 之间，使用函数进行了以下操作：

设置颜色 (`glColor3f`)

平移 (`glTranslatef`)

缩放 (`glScalef`)

绘制立方体 (`glutSolidCube`)

6. 绘制门

```
// door
glPushMatrix();
    // door blanket
    glPushMatrix();
        glColor3f( red: 0.4, green: 0.12, blue: 0.5);
        glTranslatef( x: 75.0, y: -99.9, z: -40.0);
        glBegin( mode: GL_QUADS);
        glVertex3f( x: 25.0, y: 0.0, z: 35.0);
        glVertex3f( x: 25.0, y: 0.0, z: -35.0);
        glVertex3f( x: -25.0, y: 0.0, z: -35.0);
        glVertex3f( x: -25.0, y: 0.0, z: 35.0);
        glEnd();
    glPopMatrix();
    // outdoor
    glTranslatef( x: 99.9, y: -50.0, z: -40.0);
    glColor3f( red: 0.0, green: 0.0, blue: 0.0);
    glBegin( mode: GL_QUADS);
        glVertex3f( x: 0.0, y: -50.0, z: 35.0);
        glVertex3f( x: 0.0, y: 50.0, z: 35.0);
        glVertex3f( x: 0.0, y: 50.0, z: -35.0);
        glVertex3f( x: 0.0, y: -50.0, z: -35.0);
    glEnd();
    // the door
    glColor3f( red: 0.8, green: 0.6, blue: 0.4);
```

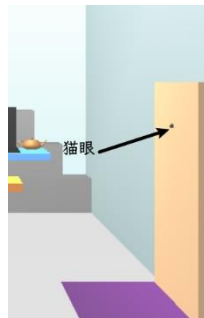
这段代码是 OpenGL 中绘制门和门上饰物的代码。代码中使用了 OpenGL 中的几何变换函数，例如 `glTranslatef`，`glRotatef` 和 `glScalef`，以及绘制图形的函数，例如 `glutSolidCube` 和 `glutSolidSphere`。

首先，使用了 `glPushMatrix` 和 `glPopMatrix` 函数对当前的变换矩阵进行保存和恢复，以确保对门的绘制不影响其他图形的绘制。

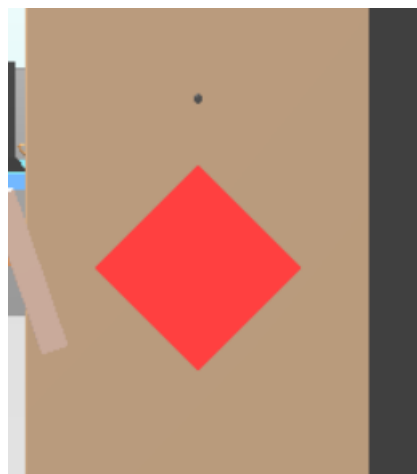
然后，使用 `glTranslatef` 函数平移门的位置，使用 `glScalef` 函数改变门的大小，并使用 `glRotatef` 函数设置门的旋转角度。

随后，绘制了一个立方体作为门，使用 `glColor3f` 函数设置了门的颜色。

接下来，绘制了猫眼，使用 `glutSolidSphere` 函数绘制了两个圆球，并使用 `glTranslatef` 函数移动了它们的位置。



最后，绘制了门上的春联，使用 `glBegin` 和 `glEnd` 函数绘制一个四边形，并使用 `glRotatef` 函数设置了它的旋转角度。



7. 电脑


```

// desk & computer & seat & teapot
glPushMatrix();
// desk
glColor3f( red: 0.22, green: 0.52, blue: 1.0);
glTranslatef( x: 20.0, y: -50.0, z: -225.0);
glScalef( x: 80.0, y: 4.0, z: 50.0);
glutSolidCube( size: 1.0);
glScalef( x: 1.0 / 60.0, y: 0.25, z: 0.02);
// computer
glTranslatef( x: -5.0, y: 2.0, z: 0.0);
glColor3f( red: 0.0, green: 0.0, blue: 0.0);
glScalef( x: 40.0, y: 2.0, z: 30.0);
glutSolidCube( size: 1.0);
glScalef( x: 1.0 / 40.0, y: 0.5, z: 1.0 / 30.0);
glTranslatef( x: 0.0, y: 15.0, z: -20.0);
glScalef( x: 40.0, y: 30.0, z: 2.0);
glColor3f( red: 0.0, green: 0.0, blue: screen); // 可设特效
glutSolidCube( size: 1.0);
glScalef( x: 1.0 / 40.0, y: 1.0 / 30.0, z: 0.5);
// teapot
glTranslatef( x: 27.0, y: -10.0, z: 20.0);
glColor3f( red: 0.5, green: 0.3, blue: 0.1);
glutSolidTeapot( size: 5.0);

```

Idle 函数

Idle 函数根据用户从键盘的输入，对参数进行相应更改，然后将更改同步到窗口

```

if (option == 1) {
    if (screen <= 1.0) {
        Sleep( dwMilliseconds: 100);
        screen += 0.1; // 屏幕变亮
    }
}
if (option == 2) {
    if (screen >= 0.0) {
        Sleep( dwMilliseconds: 100);
        screen -= 0.1; // 屏幕变暗
    }
}
if (option == 3) {
    if (arm <= 130.0) {
        Sleep( dwMilliseconds: 5);
        arm += 1; // 手臂摆动
    }
}

```

option 是从键盘获取到的值

```

    }
    if (option == 5) {
        if (xiaom >= -60.0) {
            if (doorOpen == 1) {
                Sleep( dwMilliseconds: 5);
                xiaom -= 1;  移动小明
            }
        }
    }
    if (option == 6) {
        if (doorOpen == 1) {
            if (xiaom <= 150.0) {
                Sleep( dwMilliseconds: 5);
                xiaom += 1;
            }
        }
        else {
            if (xiaom <= 15.0) {
                Sleep( dwMilliseconds: 5);
                xiaom += 1;
            }
        }
    }
}

```

Xiaom 变量放在这控制移动

```
glTranslatef( x: xiaom, y: -10.0, z: -30.0); // 可设动画
```

```

    }
    if (option == 7) {
        doorOpen = 1;
        if (door <= 90.0) {
            Sleep( dwMilliseconds: 5);
            door += 1;
        }
    }
    if (option == 8) {
        doorOpen = 0;
        if (door >= 0.0) {
            Sleep( dwMilliseconds: 5);
            door -= 1;
        }
    }
    if (option == 9) {
        l1 = 0.25;
        l1 = 1.0;
    }
    if (option == 10) {
        l1 = 0.0;
        l1 = 0.0;
    }
}

```

通过door变量控制门的开关

```
glRotatef( angle: door, x: 0.0, y: -1.0, z: 0.0); // 可设动画
```

变量名	类型	说明
screen	float	屏幕大小
option	int	选项，决定程序如何运行
arm	float	机械臂的大小
alice	float	某个物体的位置
doorsOpen	int	门是否打开，1代表打开，0代表关闭
door	float	门的大小
l1	float	某个物体的位置
l2	float	某个物体的位置

六、实验心得

1. 每次 `glPushMatrix()` 和 `glPopMatrix()` 之间的变换都是临时的，在 `glPopMatrix()` 调用之后，模型观察矩阵回到了它以前的状态。这样可以方便地在同一位置绘制多个物体，而不影响其他物体的渲染。

2. 有关动画的实现心得

`idle` 函数实现了对机械臂状态的不断更新，从而实现机械臂的动态操作。

在该函数中，通过 `if` 语句和 `switch` 语句，根据不同的操作选项（`option`）来实现不同的动作。例如，当 `option` 的值为 1 时，执行屏幕上升的操作；当 `option` 的值为 7 时，执行门开启的操作。

在实现每一个动作时，都需要判断当前机械臂的状态是否满足条件，以确保机械臂的操作安全。例如，在实现机械臂上升操作时，需要判断机械臂当前的角度（`arm`）是否小于等于 130.0。

整段代码中使用了一个 `Sleep` 函数，该函数用于实现动作的暂停，以便让用户更好地观察动作的效果。此外，整段代码还使用了一个 `glutPostRedisplay` 函数，该函数用于刷新显示，以保证用户可以看到机械臂的实时状态。

总之，这段代码是一段非常有用的代码，它通过结合了 `if` 语句、`switch` 语句、`Sleep` 函数和 `glutPostRedisplay` 函数等技术，实现了对机械臂操作的模拟。

七、完整代码

```

1 #include <stdlib.h>
2 #include<stdio.h>
3 #include<windows.h>
4 #include <GL/glut.h>
5 const GLdouble FRUSTDIM = 100.0f;
6 GLfloat door = 0.0;
7 GLfloat sticker = 45.0;
8 GLfloat screen = 0.0;
9 GLfloat arm = 20.0;

```

```

10 GLfloat xiaom = -60.0;
11 int option = 0;
12 int doorOpen = 0;
13 float l1 = 0.25;
14 float l1 = 1.0;
15
16 void init(void) // All Setup For OpenGL Goes Here
17 {
18     glEnable(GL_NORMALIZE); // 不加此项会出现放大后物体变暗问题
19     glEnable(GL_DEPTH_TEST);
20     glEnable(GL_LIGHTING);
21     glEnable(GL_LIGHT0);
22     glEnable(GL_LIGHT1);
23     glFrontFace(GL_CCW);
24     glEnable(GL_COLOR_MATERIAL);
25 }
26
27 void display(void) // Here's Where We Do All The Drawing
28 {
29     glPushMatrix();
30     //glClearColor(1.0, 1.0, 1.0, 0.0);
31     glClear(GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT); //必须放在
display 中否则动画会有 bug
32     // TODO:
33     // Place light source here
34     GLfloat lightpos[] = { 0.f, 100.f, -450.f, l1 }; // 可设动
画
35     GLfloat light0_mat1[] = { 0.2, 0.2, 0.2, 0.3f };
36     GLfloat light0_diff[] = { 1.0, 1.0, 1.0, 0.3 };
37     GLfloat lightpos2[] = { 0.0,0.0,0.0,l1 };
38     GLfloat light1_mat1[] = { 0.1,0.1,0.1,0.3 };
39     GLfloat light1_diff[] = { 0.1,0.1,0.1,0.3 };
40     glLightfv(GL_LIGHT0, GL_POSITION, lightpos);
41     glLightfv(GL_LIGHT0, GL_AMBIENT, light0_mat1);
42     glLightfv(GL_LIGHT0, GL_DIFFUSE, light0_diff);
43     glLightfv(GL_LIGHT1, GL_POSITION, lightpos2);
44     glLightfv(GL_LIGHT1, GL_AMBIENT, light1_mat1);
45     glLightfv(GL_LIGHT1, GL_DIFFUSE, light1_diff);
46
47     GLfloat no_mat[] = { 0.0,0.0,0.0,1.0 };
48     GLfloat mat_ambient[] = { 0.2,0.2,0.2,1.0 };
49     GLfloat mat_diffuse[] = { 0.9,0.9,0.9,1.0 };
50     GLfloat mat_specular[] = { 0.3,0.3,0.3,1.0 };
51     GLfloat high shininess[] = { 20.0 };

```

```
52  GLfloat high_mat[] = { 11,11,11,1.0 };
53
54  glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambient);
55  glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diffuse);
56  glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
57  glMaterialfv(GL_FRONT, GL_SHININESS, high_shininess);
58  glMaterialfv(GL_FRONT, GL_EMISSION, high_mat);
59
60  glTranslatef(0.0, 0.0, -300.0);
61  // TODO:
62  // Draw walls and objects here
63  glPushMatrix();
64  static GLfloat wall_mat[] = { 1.f, 1.f, 1.f, 1.f };
65  glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, wall_mat);
66
67  // back wall
68  glColor3f(0.5, 0.6, 0.6);
69  glBegin(GL_QUADS);
70  glNormal3f(0.0, 0.0, 1.0);
71  glVertex3f(100.0, 100.0, -300.0);
72  glVertex3f(-100.0, 100.0, -300.0);
73  glVertex3f(-100.0, -100.0, -300.0);
74  glVertex3f(100.0, -100.0, -300.0);
75  glEnd();
76
77  // bottom wall
78  glColor3f(0.5, 0.5, 0.5);
79  glBegin(GL_QUADS);
80  glNormal3f(0.0, 1.0, 0.0);
81  glVertex3f(100.0, -100.0, 0.0);
82  glVertex3f(100.0, -100.0, -300.0);
83  glVertex3f(-100.0, -100.0, -300.0);
84  glVertex3f(-100.0, -100.0, 0.0);
85  glEnd();
86
87  // top wall
88  glColor3f(0.7, 0.7, 0.9);
89  glBegin(GL_QUADS);
90  glNormal3f(0.0, -1.0, 0.0);
91  glVertex3f(100.0, 100.0, 0.0);
92  glVertex3f(-100.0, 100.0, 0.0);
93  glVertex3f(-100.0, 100.0, -300.0);
94  glVertex3f(100.0, 100.0, -300.0);
95  glEnd();
```

```
96
97 // left wall
98 glColor3f(0.5, 0.6, 0.6);
99 glBegin(GL_QUADS);
100 glNormal3f(1.0, 0.0, 0.0);
101 glVertex3f(-100.0, -100.0, 0.0);
102 glVertex3f(-100.0, -100.0, -300.0);
103 glVertex3f(-100.0, 100.0, -300.0);
104 glVertex3f(-100.0, 100.0, 0.0);
105 glEnd();
106
107 // right wall
108 glColor3f(0.5, 0.6, 0.6);
109 glBegin(GL_QUADS);
110 glNormal3f(-1.0, 0.0, 0.0);
111 glVertex3f(100.0, -100.0, 0.0);
112 glVertex3f(100.0, 100.0, 0.0);
113 glVertex3f(100.0, 100.0, -300.0);
114 glVertex3f(100.0, -100.0, -300.0);
115 glEnd();
116
117 // bed
118 glPushMatrix();
119 glColor3f(0.7, 0.7, 0.5);
120 glTranslatef(-65.0, 20.0, -175.0);
121 glScalef(3.5, 1.0, 12.5);
122 glutSolidCube(20.0);
123 glPopMatrix();
124
125 // wardrobe
126 glPushMatrix();
127 glColor3f(0.6, 0.4, 0.2);
128 glTranslatef(-65.0, -45.0, -175.0);
129 glScalef(14.0, 22.0, 50.0);
130 glutSolidCube(5.0);
131 glPopMatrix();
132
133 // mirror
134 glPushMatrix();
135 glTranslatef(-65.0, -45.0, -49.0);
136 glColor3f(0.79, 0.82, 0.98);
137 glScalef(60.0, 100.0, 2.0);
138 glutSolidCube(1.0);
139 glPopMatrix();
```

```
140
141 // xiaoming
142 glPushMatrix();
143 glTranslatef(xiaom, -10.0, -30.0); // 可设动画
144 // arms
145 glColor3f(0.9, 0.7, 0.6);
146 glPushMatrix();
147 glTranslatef(-12.5, -25.0, 0.0);
148 glRotatef(arm, 0.0, 0.0, -1.0); // 可设动画
149 glTranslatef(0.0, -15.0, 0.0);
150 glScalef(5.0, 30.0, 5.0);
151 glutSolidCube(1.0);
152 glPopMatrix();
153 glPushMatrix();
154 glTranslatef(12.5, -25.0, 0.0);
155 glRotatef(arm, 0.0, 0.0, 1.0);
156 glTranslatef(0.0, -15.0, 0.0);
157 glScalef(5.0, 30.0, 5.0);
158 glutSolidCube(1.0);
159 glPopMatrix();
160 // head
161 glTranslatef(0.0, -10.0, 0.0);
162 glColor3f(0.9, 0.7, 0.6);
163 glutSolidSphere(8.0, 50, 50);
164 // neck
165 glTranslatef(0.0, -10.0, 0.0);
166 glScalef(8.0, 10.0, 10.0);
167 glutSolidCube(1.0);
168 // body
169 glColor3f(0.2, 0.2, 0.2);
170 glScalef(0.125, 0.1, 0.1);
171 glTranslatef(0.0, -18.0, 0.0);
172 glScalef(25.0, 35.0, 15.0);
173 glutSolidCube(1.0);
174 // legs
175 glColor3f(0.3, 0.3, 0.3);
176 glScalef(1.0 / 25.0, 1.0 / 35.0, 1.0 / 15.0);
177 glTranslatef(-9.0, -35.0, 0.0);
178 glScalef(7.0, 35.0, 15.0);
179 glutSolidCube(1.0);
180 glScalef(1.0 / 7.0, 1.0 / 35.0, 1.0 / 15.0);
181 glTranslatef(18.0, 0.0, 0.0);
182 glScalef(7.0, 35.0, 15.0);
183 glutSolidCube(1.0);
```

```
184     glPopMatrix();
185
186     // stairs
187     glPushMatrix();
188     glTranslatef(-30.0, -100.0, -275.0);
189     glColor3f(0.3, 0.3, 0.3);
190     glPushMatrix();
191     glTranslatef(65.0, 13.0, -13.0);
192     glScalef(65.0, 13.0, 13.0);
193     glutSolidCube(2.0);
194     glPopMatrix();
195     glColor3f(0.3, 0.3, 0.3);
196     glPushMatrix();
197     glTranslatef(54.0, 39.0, -13.0);
198     glScalef(54.0, 13.0, 13.0);
199     glutSolidCube(2.0);
200     glPopMatrix();
201     glColor3f(0.3, 0.3, 0.3);
202     glPushMatrix();
203     glTranslatef(43.0, 65.0, -13.0);
204     glScalef(43.0, 13.0, 13.0);
205     glutSolidCube(2.0);
206     glPopMatrix();
207     glColor3f(0.3, 0.3, 0.3);
208     glPushMatrix();
209     glTranslatef(32.0, 91.0, -13.0);
210     glScalef(32.0, 13.0, 13.0);
211     glutSolidCube(2.0);
212     glPopMatrix();
213     glColor3f(0.3, 0.3, 0.3);
214     glPushMatrix();
215     glTranslatef(21.0, 117.0, -13.0);
216     glScalef(21.0, 13.0, 13.0);
217     glutSolidCube(2.0);
218     glPopMatrix();
219     glPopMatrix();
220
221     // top light
222     glPushMatrix();
223     glColor3f(1.0, 1.0, 0.5);
224     glTranslatef(0.0, 94.0, -150.0);
225     glScalef(80.0, 4.0, 80.0);
226     glutSolidCube(1.0);
227     glPopMatrix();
```



```

228     glPushMatrix();
229     glColor3f(0.6, 0.5, 0.5);
230     glTranslatef(0.0, 99.0, -150.0);
231     glScalef(100.0, 2.0, 100.0);
232     glutSolidCube(1.0);
233     glPopMatrix();
234
235     // door
236     glPushMatrix();
237     // door blanket
238     glPushMatrix();
239     glColor3f(0.4, 0.12, 0.5);
240     glTranslatef(75.0, -99.9, -40.0);
241     glBegin(GL_QUADS);
242     glVertex3f(25.0, 0.0, 35.0);
243     glVertex3f(25.0, 0.0, -35.0);
244     glVertex3f(-25.0, 0.0, -35.0);
245     glVertex3f(-25.0, 0.0, 35.0);
246     glEnd();
247     glPopMatrix();
248     // outdoor
249     glTranslatef(99.9, -50.0, -40.0);
250     glColor3f(0.0, 0.0, 0.0);
251     glBegin(GL_QUADS);
252     glVertex3f(0.0, -50.0, 35.0);
253     glVertex3f(0.0, 50.0, 35.0);
254     glVertex3f(0.0, 50.0, -35.0);
255     glVertex3f(0.0, -50.0, -35.0);
256     glEnd();
257     // the door
258     glColor3f(0.8, 0.6, 0.4);
259     glTranslatef(0.0, 0.0, -35.0);
260     glRotatef(door, 0.0, -1.0, 0.0); // 可设动画
261     glTranslatef(0.0, 0.0, 35.0);
262     glScalef(6.0, 100.0, 70.0);
263     glutSolidCube(1.0);
264     // cat eye
265     glScalef(1.0 / 6.0, 0.01, 1.0 / 70.0);
266     glTranslatef(4.0, 30.0, 0.0);
267     glColor3f(0.1, 0.1, 0.1);
268     glutSolidSphere(1.0, 10, 10);
269
270     glTranslatef(-8.0, 0.0, 0.0);
271     glutSolidSphere(1.0, 10, 10);

```

```

272     glTranslatef(12.0, 0.0, 0.0);
273
274     // door sticker
275     glTranslatef(-4.5, -35.0, 0.0);
276     glColor3f(1.0, 0.0, 0.0);
277     glRotatef(sticker, 1.0, 0.0, 0.0);    // 可设动画
278     glBegin(GL_QUADS);
279     glVertex3f(0.0, 15.0, 15.0);
280     glVertex3f(0.0, 15.0, -15.0);
281     glVertex3f(0.0, -15.0, -15.0);
282     glVertex3f(0.0, -15.0, 15.0);
283     glEnd();
284     glPopMatrix();
285
286     // desk & computer & seat & teapot
287     glPushMatrix();
288     // desk
289     glColor3f(0.22, 0.52, 1.0);
290     glTranslatef(20.0, -50.0, -225.0);
291     glScalef(80.0, 4.0, 50.0);
292     glutSolidCube(1.0);
293     glScalef(1.0 / 60.0, 0.25, 0.02);
294     // computer
295     glTranslatef(-5.0, 2.0, 0.0);
296     glColor3f(0.0, 0.0, 0.0);
297     glScalef(40.0, 2.0, 30.0);
298     glutSolidCube(1.0);
299     glScalef(1.0 / 40.0, 0.5, 1.0 / 30.0);
300     glTranslatef(0.0, 15.0, -20.0);
301     glScalef(40.0, 30.0, 2.0);
302     glColor3f(0.0, 0.0, screen);    // 可设特效
303     glutSolidCube(1.0);
304     glScalef(1.0 / 40.0, 1.0 / 30.0, 0.5);
305     // teapot
306     glTranslatef(27.0, -10.0, 20.0);
307     glColor3f(0.5, 0.3, 0.1);
308     glutSolidTeapot(5.0);
309     // seat
310     glTranslatef(-27.0, -50.0, 60.0);
311     glColor3f(1.0, 0.5, 0.13);
312     glRotatef(-90.0, 1.0, 0.0, 0.0);
313     GLUQuadricObj *g_text;
314     g_text = gluNewQuadric();
315     gluCylinder(g_text, 10.0, 10.0, 30.0, 50, 4);

```

```

316     glRotatef(90.0, 1.0, 0.0, 0.0);
317     glTranslatef(0.0, 30.0, 0.0);
318     glScalef(35.0, 5.0, 35.0);
319     glutSolidCube(1.0);
320     glPopMatrix();
321     glPopMatrix();
322
323     // TODO:
324     // Add animation here
325     //glLoadIdentity();
326     glPopMatrix();
327     glFlush();
328     glutSwapBuffers();
329 }
330
331
332 void reshape(int w, int h) // Resize the GL Window. w=width,
h=height
333 {
334     glViewport(0, 0, (GLsizei)w, (GLsizei)h);
335     glMatrixMode(GL_PROJECTION);
336     glLoadIdentity();
337     glFrustum(-FRUSTDIM, FRUSTDIM, -FRUSTDIM, FRUSTDIM, 300.,
800.);
338     glMatrixMode(GL_MODELVIEW);
339     glLoadIdentity();
340 }
341
342 void keyboard(unsigned char key, int x, int y) // Handle the
keyboard events here
343 {
344     switch (key)
345     {
346         case '\033': //press 'esc' to quit
347             exit(0);
348             break;
349
350         // TODO:
351         // Add keyboard control here
352         case '1':
353             printf("computer power on\n");
354             option = 1;
355             break;
356         case '2':

```

```

357         printf("computer power off\n");
358         option = 2;
359         break;
360     case '3':
361         printf("xiaoming hands up\n");
362         option = 3;
363         break;
364     case '4':
365         printf("xiaoming hands down\n");
366         option = 4;
367         break;
368     case '5':
369         printf("xiaoming walk left\n");
370         option = 5;
371         break;
372     case '6':
373         printf("xiaoming walk right\n");
374         option = 6;
375         break;
376     case '7':
377         printf("open the door");
378         option = 7;
379         break;
380     case '8':
381         printf("close the door\n");
382         option = 8;
383         break;
384     case '9':
385         printf("turn on the light\n");
386         option = 9;
387         break;
388     case '0':
389         printf("turn off the light\n");
390         option = 10;
391         break;
392     default:
393         printf("error\n");
394 }
395 }
396
397 void idle()
398 {
399     if (option == 1) {
400         if (screen <= 1.0) {

```

```
401         Sleep(100);
402         screen += 0.1;
403     }
404 }
405 if (option == 2) {
406     if (screen >= 0.0) {
407         Sleep(100);
408         screen -= 0.1;
409     }
410 }
411 if (option == 3) {
412     if (arm <= 130.0) {
413         Sleep(5);
414         arm += 1;
415     }
416 }
417 if (option == 4) {
418     if (arm >= 20.0) {
419         Sleep(5);
420         arm -= 1;
421     }
422 }
423 if (option == 5) {
424     if (xiaom >= -60.0) {
425         if (doorOpen == 1) {
426             Sleep(5);
427             xiaom -= 1;
428         }
429     }
430 }
431 if (option == 6) {
432     if (doorOpen == 1) {
433         if (xiaom <= 150.0) {
434             Sleep(5);
435             xiaom += 1;
436         }
437     }
438     else {
439         if (xiaom <= 15.0) {
440             Sleep(5);
441             xiaom += 1;
442         }
443     }
444 }
```

```

445     if (option == 7) {
446         doorOpen = 1;
447         if (door <= 90.0) {
448             Sleep(5);
449             door += 1;
450         }
451     }
452     if (option == 8) {
453         doorOpen = 0;
454         if (door >= 0.0) {
455             Sleep(5);
456             door -= 1;
457         }
458     }
459     if (option == 9) {
460         l1 = 0.25;
461         l1 = 1.0;
462     }
463     if (option == 10) {
464         l1 = 0.0;
465         l1 = 0.0;
466     }
467     glutPostRedisplay();
468 }
469
470 int main(int argc, char** argv)
471 {
472     /*Initialization of GLUT Library */
473     glutInit(&argc, argv);
474     glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH);
475
476     /*Create a window with title specified */
477     glutInitWindowSize(600, 600);
478     glutInitWindowPosition(100, 100);
479     glutCreateWindow("图形学综合实验");    // 修改 bug, createWindow
前应设置好窗口大小和位置
480
481     init(); /*not GLUT call, initialize several parameters */
482
483     /*Register different CALLBACK function for GLUT to response
484     with different events, e.g. window sizing, mouse click or
485     keyboard stroke */
486     glutReshapeFunc(reshape);
487     glutDisplayFunc(display);

```

```
488     glutKeyboardFunc(keyboard);
489     glutIdleFunc(idle);
490
491     /*Enter the GLUT event processing loop which never returns.
492     it will call different registered CALLBACK according
493     to different events. */
494     glutMainLoop();
495 }
```